МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Дніпропетровський національний університет залізничного транспорту імені академіка В. Лазаряна

Кафедра «Комп'ютерні інформаційні технології»

ЯКІСТЬ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ТА ТЕСТУВАННЯ

Методичні вказівки до виконання лабораторних робіт

Укладачі: В. І. Шинкаренко,
О. С. Куроп'ятник,
Г. В. Забула,
Д. О. Пєтін,
Є. В. Лукін

Для студентів IV курсу денної форми навчання спеціальності 121 «Інженерія програмного забезпечення»

Дніпро 2018

УДК 004.05

Укладачі:
проф. *Шинкаренко Віктор Іванович*
ас. *Куроп'ятник Олена Сергіївна*
ас. *Забула Геннадій Володимирович*
ас. *Пєтін Дмитро Олегович*
асп. *Лукін Єлисей Віталійович*

Рецензенти:
к.т.н., доц. Андрющенко В. О., ДНУЗТ
к.ф.-м.н., доц. Зайцев В. Г., ДНУ

Якість програмного забезпечення та тестування [Текст]: методичні вказівки до лабораторних робіт / уклад.: В. І. Шинкаренко, О. С. Куроп'ятник, Г. В. Забула, Д. О. Пєтін, Є. В. Лукін, Дніпропетр. нац. ун-т залізн. трансп. ім. акад. В. Лазаряна. – Д.: Вид-во ПФ «Стандарт-Сервіс», 2018. – 50 с.

Методичні вказівки призначені для студентів спеціальності 121 «Інженерія програмного забезпечення». Містять короткі теоретичні відомості для лабораторних робіт та порядок їх виконання. Призначені для теоретичної підготовки до виконання лабораторних робіт. Окремо виділений матеріал для поглибленого вивчення та контролю знань.

Іл. 5. Табл. 13. Бібліогр.: 5 назв.

© Шинкаренко В. І., Куроп'ятник О. С., Забула Г. В, Пєтін Д. О., Лукін Є. В., укладання, редагування, оригінал-макет, 2018

ЛАБОРАТОРНА РОБОТА № 1

Тема. Тестування методами білої скриньки.

Мета роботи. Отримати практичні навички тестування методами білої скриньки.

Зміст роботи:

1) ознайомитися з методами тестування програм;
2) виявити переваги та недоліки методів тестування білою скринькою;
3) підготувати програму для тестування;
4) підготувити тести методами білої скриньки;
5) виконати тестування підготовленої програми;
6) оцінити якість виконаної роботи.

Зміст звіту:

1) вид і номер роботи, дисципліна, інформація про виконавця (група, П. І. Б.);
2) тема, мета лабораторної роботи;
3) постановка задачі;
4) текст програми, яка включає функції для тестування та драйвер (програму, що подає на вхід значення для тестування та дозволяє продемонструвати та перевірити отримані результати); специфікація;
5) розроблені тести та демонстрація їх відповідності методам тестування:
   – покриття операторів;
   – покриття рішень;
   – покриття умов;
   – покриття умов та рішень;
   – комбінаторного покриття умов.
6) аналіз результатів та висновки, що включають оцінку якості виконаної роботи.

Теоретичні відомості

1. Загальні поняття теорії тестування

Тестування – процес виконання програми (або її частини) з метою перевірки її відповідності встановленим вимогам, зазначеним у специфікації, і виявлення дефектів (помилок).

Під специфікацією будемо розуміти точний, недвозначний опис дій (операцій), які має виконувати програма (або її частина), її вхідних та вихідних даних.

Помилка – дані та/або стан програми, що є результатом роботи програми і не відповідає її специфікації.

Тест – набір вхідних і вихідних даних для разового виконання програми, що задовольняє специфікації.

Хороший тест – тест, який має високу ймовірність виявлення помилок. Так, наприклад, якщо додаток має надавати доступ до інформації користувачам віком від 18 до 25 років, то тест, вхідними даними якого є вік 19 років –

хорошим, а тест, де вік – 20 років, таким не є, якщо він проходиться після першого.

Тестування полягає у динамічній перевірці поведінки програми на скінченній множині тестових даних, вибраних спеціальним чином з нескінченного вхідного простору, на відповідність встановленій очікуваній поведінці.

Тестування завжди передбачає виконання програми. Неповнота – одна з основних проблем тестування, оскільки на практиці повну множину тестів можна розглядати як нескінченну. Кількість же тестів, які можуть бути виконані в обмежені терміни, – скінченна.

Виділяють чотири рівні тестування:

– модульне (перевірка функціональних об'єктів окремо один від одного);
– інтеграційне (перевірка правильності взаємодії між програмними об'єктами, які були протестовані раніше автономно);
– тестування ПЗ (перевірка ПЗ у середовищі, що моделюється);
– системне (перевірка взаємодії ПЗ з іншими, в тому числі не програмними, компонентами системи).

Виділяють такі види тестування характеристик програмної системи:

– функціональне тестування;
– тестування безпеки;
– тестування зручності використання;
– тестування технічних характеристик;
– тестування на надійність;
– тестування продуктивності;
– тестування конфігурації;
– порівняльне тестування;
– тестування відновлення.

Методи тестування різняться підходами до проектування тестів. Традиційно методи функціонального тестування розділяють на дві категорії – чорна скринька (без доступу до тексту програми) та біла скринька (з доступом).

До методів білої скриньки відносяться:

– покриття операторів;
– покриття рішень;
– покриття умов;
– покриття умов та рішень;
– комбінаторне покриття умов.

Метод покриття операторів передбачає підбір таких наборів вхідних даних, при яких кожен оператор програми виконається хоча б один раз. Результати тестування можуть бути оформлені у вигляді таблиці з вказівкою номерів операторів (присвоєні за порядком заздалегідь) по вертикалі та номерів тестів по горизонталі. Відмітка про виконання оператору ставиться на перетині відповідного рядка та стовпця, що відповідає тесту, який виконується.

При тестуванні під рішенням будемо розуміти набір умов та їх значень, що визначають хід виконання програми. До набору умов слід включити умови всіх керуючих структур: циклів усіх видів (for, while, do-while для мови

C/C++) та розгалужень (if, switch). Приклади умов на мові С: a>b, current==true, a>b || a!=0, b==4. При покритті рішень кожен напрям (кожен можливий варіант з набору умов та значень) має бути реалізовано хоча б один раз. В цьому методі, а також наступних необхідно також враховувати покриття операторів.

Метод покриття умов передбачає створення такого набору тестів, за яким кожна умова виконається та не виконається хоча б один раз. Результати можуть бути представлені аналогічно методу покриття операторів.

При тестуванні методом покриття рішень та умов можливі результати кожної умови у рішенні отримані принаймні один раз і всі рішення виконались хоча б один раз.

У методі комбінаторного покриття умов необхідно протестувати всі можливі комбінації умов.

2. Тестування білою скринькою

Розглянемо приклад тестування функцій методами білої скриньки.

Розглянемо функцію ітераційного додавання елемента в двозв'язний список.

Специфікація

Функція 1. Функція ітераційного додавання нового елемента в упорядкований двозв'язний список.

Заголовок void addmid(List*& first,int element)

Функція приймає на вхід у явному виді:

– first – вказівник на початок списку;

– element – цілочислене значення інформаційної частини нового елемента.

Функція не приймає параметри в неявному вигляді.

У явному вигляді функція не повертає значень. У неявному – список, що включає в себе новий елемент.

Текст програми

Функція 1.

```
/* Функція додавання елементу в упорядкований список
first – вказівник на початок списку
element – значення елементу, який треба додати */
   void addmid(List*& first, int element)
   {
     List* tmp = nullptr;//допоміжна змінна

     //P1. Додати елемент до списку
     //P1.1. Додати елемент у вже існуючий список
     if (first != nullptr)
     {
          tmp = first;
          //P1.1.1. Додати елемент в початок списку
          if (first->inf > element)
          {
               List* new_first = new List;
               new_first->inf = element;
```

```
                new_first->next = first;
                first->prev = new_first;
                first = new_first;
                first->prev = nullptr;
           }
           //P1.1.2. Додати елемент в середину списку
           else
           {
                //P1.1.2.1 Знайти позицію для вставки
                while ((tmp->next) && (tmp->next->inf < element))
                {
                     tmp = tmp->next;
                }
                //P1.1.2.2. Додати в середину
                if (tmp->next != nullptr)
                {
                     List *el = new List;
                     el->inf = element;
                     el->prev = tmp;
                     el->next = tmp->next;
                     tmp->next->prev = el;
                     tmp->next = el;
                     tmp = el;
                }
                //P1.1.2.3. Додати в кінець
                else
                {
                     List *el = new List; /*новий останній еле-
мент*/
                     el->inf = element;
                     el->prev = tmp;
                     el->next = nullptr;
                     tmp->next = el;
                }
           }
      }
      //P1.2. Додати елемент в порожній список
      else
      {
           first = new List;
           first->inf = element;
           first->next = nullptr;
      }
   }
```

Для тестування функції наведено опис структури списку

```
struct  List
{
     int inf;
     List* next;
     List* prev;
};
```

Для перевірки роботи функції напишемо програму-драйвер, яка дозволяє задавати значення нових елементів списку, формувати список, а також переглядати його елементи в прямому та зворотному напрямі для перевірки коректності значень вказівників на наступний та попередній елементи.

```
/*Функція виведення елементів списку у консоль
current – вказівник на початок списку*/
void show(List* current)
{
	//P1. Вивести всі елементи у прямому напрямі
	// Доки є елементи виводити
	while (current != nullptr)
	{
		//P1.1. Вивести елемент
		cout << current->inf << "\t";
		//P1.2. Перейти до наступного
		current = current->next;
	}

}
/*Функція виведення елементів списку в зворотному напрямі в консоль
current - вказівник на початок списку*/
void rshow(List* current)
{
	//P1. Вивести елементи в зворотному напрямі
	//P1.1. Перейти до останнього елементу
	while (current->next != nullptr)
	{
		current = current->next;
	}
	//P1.2. Виводити доки є елементи
	while (current != nullptr)
	{
		//P1.2.1. Вивести елемент
		cout << current->inf << "\t";
		//P1.2.2. Перейти до попереднього
		current = current->prev;
	}
}

/*Головна функція програми-драйвера для тестування */
void main()
{
	//P1. Продемонструвати додавання елементів до двозв’язного списку
	List* number_list=nullptr; //список цілих чисел
	char choice; //для організації меню
	do
	{
		//P1.1. Вивести меню
		cout << "1. Add element" << endl;
		cout << "2. Show list" << endl;
```

```
          cout << "3. Revers show list" << endl;
          cout << "0. Exit"<<endl;
          cout << "Enter your choice: ";
          cin >> choice;
          cout << endl;
          //P1.2. Обробити обраний пункту
          switch (choice)
          {
               //P1.2.1. Додати елемент
          case '1':
          {
               int new_value;
               cout<< "Enter the value of new element: ";
               cin >> new_value;
               addmid(number_list, new_value);
               cout << "Done!" << endl;
               break;
          }
               //P1.2.2. Вивести елементи, щоб перевірити коректність значень вказівника на наступний елемент
          case '2':
          {
               if (number_list != nullptr)
               {
                    cout << "Elements: ";
                    show(number_list);
               }
               else
               {
                    cout << "List is empty";
               }
               cout << endl;
               break;
          }
               //P1.2.2. Вивести елементи в зворотному, щоб перевірити коректність значень вказівника на попередній елемент
          case '3':
          {
               if (number_list != nullptr)
               {
                    cout << "Elements (reverse order): ";
                    rshow(number_list);
               }
               else
               {
                    cout << "List is empty";
               }
               cout << endl;
               break;
          }
          }

     } while (choice != '0');
```

```
    system("pause");
}
```

Тестування функції 1

Тести:

1. Список порожній, додаємо елемент, рівний 1.
   Вхід: first =null, element=1
   Вихід: first ={1}
2. У списку є n елементів, додаємо в початок
   Вхід: first ={1,2,3}, element=0
   Вихід: first ={0,1,2,3}
3. У списку є n елементів, додаємо в середину
   Вхід: first ={1,2,3, 5}, element=4
   Вихід: first ={1,2,3,4,5}
4. У списку є n елементів, додаємо в кінець
   Вхід: first ={1,2,3}, element=5
   Вихід: first ={1,2,3,5}

Результати тестування методом покриття умов наведені в табл. 1. Тут і далі умовою «tmp->next» перевіряться наявність у списку наступного елементу.

Виконаємо тестування методом покриття рішень. Результати тестування наведені в табл. 2. Значення вхідних і вихідних даних аналогічні методу покриття умов.

Таблиця 1

Таблиця покриття умов

Умова / Тест	first!=nullptr		first >inf>element		tmp->next		tmp->next ->inf <element		tmp->next !=nullptr	
	+	-	+	-	+	-	+	-	+	-
1		*								
2	*		*							
3	*			*	*		*	*	*	
4	*			*	*	*	*			*

Таблиця 2

Таблиця покриття рішень

Напрям / Тест	first!=nullptr		first -> inf>element		tmp->next && tmp->next-> enf>element		tmp->next !=nullptr	
	+	-	+	-	+	-	+	-
1		*						
2	*		*					
3	*			*	*	*	*	
4	*			*	*	*		*

Теоретичні відомості для поглибленого вивчення

Методології проектування тестів можуть бути об'єднані в стратегію. Стратегія визначає якими методами і в якій послідовності виконувати тестування різних частин програми.

Вхідними даними для розробки стратегії є:

– термін виконання тестування;
– виробничі ресурси, включаючи людські (для підготовки та виконання тестів);
– наявні засоби тестування;
– вимоги до надійності.

При розробці стратегії необхідно врахувати час розробки одного тесту та ресурси, необхідні на його виконання.

Правила розробки стратегії:

– майже завжди мають включатися методи еквівалентних розбиттів, граничних умов та припущення про помилку;
– якщо програма має складну логічну специфікацію треба застосувати метод функціональних діаграм;
– обов'язково необхідно включити хоча б один з методів білої скриньки, який саме – визначається за вхідними умовами.

На основі стратегії розробляється план тестування, в якому визначаються які модулі, якими методами, ким та у які строки будуть протестовані.

Тестування багатомодульних програм передбачає тестування окремих програмних сутностей (підпрограм, процедур, функцій). Таке тестування має ряд переваг:

– можливість керування комбінаторикою тестування;
– полегшення процесу налагодження програми;
– паралелізм тестування та розробки в цілому.

Процес тестування включає три аспекти:

– спосіб побудови набору тестів;
– порядок тестування модулів та збору програми;
– практичні рекомендації щодо реалізації тестування.

При розробці тестів використовується специфікація модулю та його текст.

Порядок тестування модулів можу бути таким:

– зверху до низу (передбачає розробку «заглушок», які імітують дії процедур);
– знизу вгору;
– монолітний (кожен модуль тестується окремо зі своїми драйверами та «заглушками», а потім все разом).

Переваги та недоліки порядків тестування наведено в табл. 3.

Таблиця 3

Аналіз порядку тестування

Порядок	Переваги	Недоліки
Зверху до низу	Тестування може починатися майже від самого початку кодування	Необхідно розробляти «заглушки»
	Багато модулів проходять тестування декілька разів	Часом складно створити відповідні вхідні умови для тестування модулю
	Спрощення локалізації помилки	
	Постійно є працююча, протестована програма	
Знизу вгору	Багато модулів проходять тестування декілька разів	Додаткові роботи з розробки драйверів
	Спрощення локалізація помилки	Відсутня працююча, протестована програма до завершення розробки
	Тестування починається на більш ранніх стадіях роботи над проектом	Стимулювання незавершеності тестування
Монолітне	Розпаралелення роботи	Додаткові витрати часу на суміщення модулів

Завдання

Підготувати дві чи більше функцій для тестування. Функції повинні містити не менше шести керуючих конструкцій (розгалужень і циклів). Зазначені структури мають бути вкладеними. Виконати тестування функцій методами білої скриньки.

Контрольні питання

1. Основні поняття тестування (тест, тестування, хороший тест, помилка).
2. Методи тестування білою скринькою.
3. Метод покриття операторів.
4. Метод покриття умов.
5. Метод покриття рішень.
6. Метод покриття умов і рішень.
7. Метод комбінаторного покриття умов.

Контрольні питання підвищеної складності

1. Переваги та недоліки тестування білою скринькою.
2. Оптимальна кількість тестів.

3. Види тестування характеристик програмної системи.
4. Рівні тестування програм.
5. Стратегія тестування.
6. Тестування багатомодульних програм.

ЛАБОРАТОРНА РОБОТА № 2

Тема. Тестування методами чорної скриньки.

Мета роботи. Отримати практичні навички тестування методами чорної скриньки.

Зміст роботи:

1) ознайомитися з методами тестування програм чорною скринькою;
2) виявити переваги та недоліки методів тестування чорною скриньки;
3) для програми з лабораторної роботи № 1 підготовити тести методами чорної скриньки;
4) виконати тестування програми;
5) ознайомитися з поняттями unit-тестування;
6) виконати unit-тестування функцій програми лабораторної роботи № 1;
7) оцінити якість виконаної роботи.

Зміст звіту:

1) вид і номер роботи, дисципліна, інформація про виконавця (група, П. І. Б.);
2) тема, мета лабораторної роботи;
3) постановка задачі;
4) специфікації функцій програми для тестування;
5) розроблені тести та демонстрація їх відповідності методам тестування:
   – еквівалентних розбиттів;
   – граничних умов;
   – припущення про помилку;
   – функціональних діаграм;
6) розроблені unit-тести та демонстрація їх проходження;
7) аналіз результатів та висновки, що включає оцінку якості виконаної роботи.

Теоретичні відомості

1. Загальні відомості

Методи тестування різняться підходами до проектування тестів. Традиційно методи функціонального тестування розділяють на дві категорії – чорна скринька (без доступу до тексту програми) та біла скринька (з доступом).

Методи чорної скриньки:

– еквівалентне розбиття;
– аналіз граничних значень;
– припущення про помилку;
– функціональні діаграми.

У методі еквівалентного розбиття множина значень вхідних даних функції, що утворюють її вхідний простір, розбивається на набір підмножин таким чином, що в кожну підмножину потрапляють значення, еквівалентні одне одному з точки зору їх використання в тестах для виявлення помилок. Набори даних такого класу об'єднують за принципом виявлення одних і тих же помилок: якщо набір будь-якого класу виявляє деяку помилку, то передбачається, що всі інші тести цього класу еквівалентності теж виявлять цю помилку і навпаки. Усі тести, які можуть бути побудовані на основі еквівалентних значень, представляють один «клас еквівалентності», і для тестування досить вибрати тільки один тест з класу.

Виділення класів є евристичним процесом, проте рекомендується виділяти в окремі класи допустимі та недопустимі значення деякого параметру. Також слід брати до уваги такі випадки:

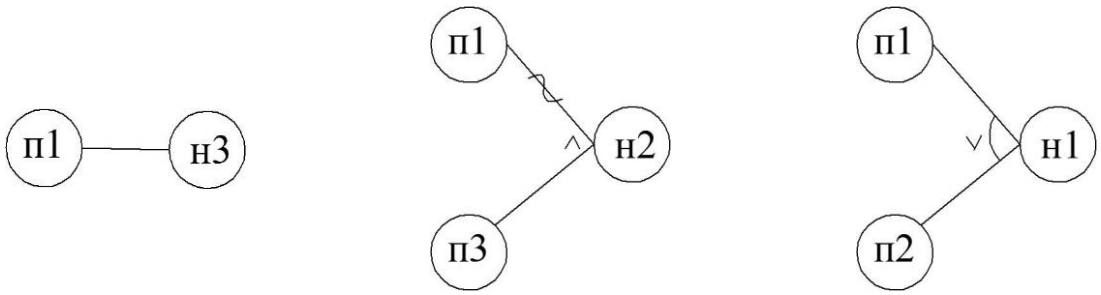
– якщо значення параметру задається діапазоном, доречно розглянути три класи: один – у межах, два з права і зліва поза межами;
– якщо специфікація описує множину вхідних значень і є підстави вважати, що кожне значення трактується особливо, то слід виділити клас під кожне значення, і один – неправильний, який не входить до множини;
– якщо задано умову «має бути», то слід розглянути два класи, один з яких задовольняє умову, інший – ні.

У методі аналізу граничних значень, що доповнює попередній метод, дані вибираються на границях вхідної області, оскільки багато відмов виникає через дефекти, пов'язані з обробкою граничних значень вхідних параметрів.

За методом функціональних діаграм спочатку ідентифікується кожна окрема функція, потім визначаються всі причини, які впливають на її результат, і всі відповідні реакції (наслідки). Наступний крок полягає в побудові графа (діаграми), що зв'язує комбінації причин з очікуваними реакціями на них. Діаграма може супроводжуватися примітками щодо обмежень та комбінацій на вхідні умови. Далі для кожного наслідку, зазначеного на графі, визначаються тестові набори шляхом перебору всіх комбінацій причин, що породжують цей наслідок.

На діаграмі для позначення умов та їх комбінацій використовують такі позначки:

– пряма лінія – тотожність (рис. 1 а, причина «п1» породжує наслідок «н3»);
– тильда «~» – заперечення (рис. 1 б);
– кон'юнкція «^» – «і» (рис. 1 б, якщо є причини «п3» і відсутня «п1», маємо наслідок «н2»);
– диз'юнкція «∨» та дуга – «або» (рис. 1 в, якщо є причина «п1» або «п2» маємо наслідок «н1»).



а *б* *в*

Рис. 1. Умовні позначення для діаграми

а – тотожність; б – заперечення та «і»; в – «або»

Також є додаткові позначення, які позначаються пунктирною лінією та буквою:

– E – взаємне виключення причин;
– I – присутня хоча б одна з причин;
– O – можлива одна і лише одна з поєднаних причин;
– R – одна причина вимагає наявність іншої;
– M – приховування (якщо причина приймає значення одиниці, то поєднаний наслідок – нуль).

2. Приклад тестування чорною скринькою

Розглянемо тестування методами чорної скриньки на прикладі функції ітераційного додавання елемента у двозв’язний список.

Специфікація

Функція 1. Функція ітераційного додавання нового елемента в упорядкований двозв’язний список.

Заголовок void addmid(List*& first,int element)

Функція приймає на вхід у явному виді:

– first – вказівник на початок списку. Може приймати значення nullptr, якщо список порожній, або значення адреси першого елементу списку;
– element– цілочислене значення інформаційної частини нового елемента.

Функція не приймає параметри в неявному вигляді

У явному вигляді функція не повертає значень. У неявному – список, що включає в себе новий елемент, а також вказівник на його початок (first).

Тестування функції 1. Розглянемо метод еквівалентних розбиттів. Функція приймає на вхід два параметра: вказівник на список і значення нового елемента списку. Виділені класи наведено у табл. 4, у дужках зазначено номери класів. Правильні класи було сформовано так, щоб значення відповідних вхідних даних задовольняли специфікацію.

Результати тестування наведені в табл. 5. В ході виконання тестів встановлено, що отримані результати за класами збігається з очікуваним.

Метод граничних умов для даного прикладу не є ефективним з огляду на те, що обмеження накладаються лише типами даних і перевіряються класами

еквівалентності 3 – 5. Проте, якщо цей метод застосувати окремо, то крім зазначених значень слід перевірити крайні границі діапазону значень типу int.

Таблиця 4

Класи еквівалентності

Вхідні умови	Правильні класи еквівалентності	Неправильні класи еквівалентності
Значення вказівника на початок списку	Вказівник рівний nullptr (1)	
	Вказівник вказує на перший елемент списку (2)	
Значення інформаційної частини нового елемента	Значення є цілим числом від -2147483648 до 2147483647 (обмеження типу int) (3)	Значення є цілим числом за межами діапазону від --2147483648 до 2147483647 (обмеження типу int) (4)
		Значення не є цілим числом (5)
Значення нового елементу по відношенню до елементів списку	Новий елемент менше усіх елементів списку (6)	
	Новий елемент більший за всі елементи списку (7)	
	Новий елемент більший/менший деяких елементів списку (8)	
	Новий елемент рівний існуючому в списку (9)	

Виконаємо тестування методом припущення про помилку. Висунемо ряд припущень:

1) якщо список порожній, то чи новий елемент, що додається, з довільним значенням стане першим? Чи буде first вказувати на нього?
2) якщо список не порожній і значення нового елементу менше за всі існуючі в списку, чи стане він при додаванні першим, а first вказувати на нього?
3) якщо список не порожній і значення нового елементу рівне значенню першого, чи буде новий елемент після додавання другим? Чи змінить значення first, що вказує на перший елемент?
4) якщо список не порожній і значення нового елементу більше за всі існуючі у списку або рівне максимальному, чи стане новий елемент після додавання останнім, і first буде вказувати на перший елемент (позиція після додавання не змінилася)?
5) якщо список не порожній і значення нового елементу більше за мінімальне існуюче у списку та менше за максимальне, то чи буде при додаванні новий елемент стояти в середині списку, а first буде вказувати на перший елемент (позиція після додавання не зміниться)?

Таблиця 5

Тести за методом еквівалентних розбиттів

Клас, що покривається	Тест	Вхід	Вихід
1	1	first= nullptr, element=1, список={}	first вказує на перший елемент, список={1}
2, 3, 8	2	first вказує на перший елемент, element=2, список={1, 3}	first вказує на перший елемент, список={1, 2 ,3}
2, 4, 7	3	first вказує на перший елемент, element=21474836471, список={1, 3}	Функція не виконується
2, 3, 5, 6	4	first вказує на перший елемент, element=4.3, список={29, 30}	first вказує на перший елемент, список={4, 29, 30}
2, 3, 9	5	first вказує на перший елемент, element=10, список={10, 30}	first вказує на перший елемент, список={10, 10, 30}

Результати перевірки подано у табл. 6.

Таблиця 6

Тести за методом припущення про помилку

Припущення	Тест	Вхід	Вихід
1	1	first =null, element=1	список = {1}, first вказує на перший елемент
2	2	список = {1,2}, element=0, first вказує на перший елемент	список = {0,1,2}, first вказує на елемент, рівний нулю. Перший елемент має значення 0.
3	3	список = {0,1,2}, element=0, first вказує на перший елемент	список ={0,0,1,2}, first вказує на перший елемент (не змінив позицію).
4	4	список = {0,0,1,2}, element=8, first вказує на перший елемент	список = {0,0,1,2,8}, first вказує на перший елемент (не змінив позицію).
5	5	список = {0,0,1,2,8}, element=4, first вказує на перший елемент	список = {0,0,1,2,4,8}, first вказує на перший елемент (не змінив позицію).

Виконаємо тестування функціональними діаграмами. Визначимо причини та наслідки (табл. 7), а також їх можливі комбінації (рис. 2).

Помітки про присутність (табл. 7) розставлено з урахуванням зав’язків на діаграмі за таким принципом: по черзі розглядаємо кожен наслідок, та виявляємо усі можливі комбінації причин, які можуть привести до нього (даного наслідку). За помітками складемо тести.

Таблиця 7

Причини та наслідки

Причини та наслідки	Умовне позначення	Значення	Помітки про присутність			
			1	2	3	4
Причини	п1	Список порожній	1			
	п2	Список не порожній		1	1	1
	п3	Новий елемент менший за всі елементи		1		
	п4	Новий елемент більший за всі елементи			1	
	п5	Новий елемент менший за деякі елементи				1
Наслідки	н1	Додавання в початок списку	1	1		
	н2	Додавання в кінець списку			1	
	н3	Додавання в середину списку				1

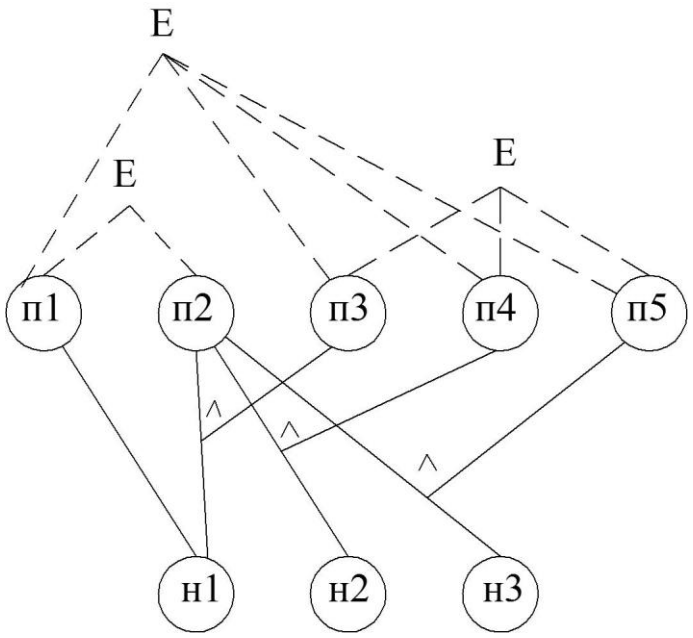


Рис. 2. Функціональна діаграма

Помітка 1 – тест 1. Вхід: first=nullptr, element=1, список={}. Вихід: first вказує на перший елемент, рівний одиниці, список={1}.

Помітка 2 – тест 2. Вхід: first вказує на перший елемент, рівний одиниці, element=0, список={1}. Вихід: first вказує на перший елемент, рівний нулю, список={0, 1}.

Помітка 3 – тест 3. Вхід: first вказує на перший елемент, рівний нулю, element=8, список={0, 1}. Вихід: first вказує на перший елемент, рівний нулю, список={0, 1, 8}.

Помітка 4 – тест 4. Вхід: first вказує на перший елемент, рівний нулю, element=3, список={0, 1, 8}. Вихід: first вказує на перший елемент, рівний нулю, список={0, 1, 3, 8}.

Подальше тестування проводиться аналогічно прикладам, розгялнутим вище, із зазначенням тестів та отриманого результату.

3. Unit-тестування

Модульне тестування, або юніт-тестування (англ. Unit testing) – процес у програмуванні, що дозволяє перевірити на коректність окремі модулі вихідного коду програми.

Мета модульного тестування – ізолювати окремі частини програми і показати, що окремо вони працездатні.

Модульний тест – це метод, який тестує метод, клас або більш великий компонент проекту (програми, додатку, рішення тощо) ізольовано від інших частин, зовнішніх систем і ресурсів, забезпечує виконання частини вашого робочого коду з очікуванням результату.

Розглянемо простий приклад створення та проходження модульного тесту.

Для створення модульних тестів у середовищі розробки виконайте такі дії (розглянуто на прикладі MS Visual Studio 2015):

1) створіть проект C# – консольний додаток;
2) напишіть клас та методи для тестування (нижче подано приклад);
3) додайте новий проект:
   - оберіть тип проекту – юніт тести;
   - додайте проект, що будете тестувати (у вікні Solution Exprorer натиснувши правою кнопкою миші на розділі посилань проекту);
   - напишіть код модульного тесту.

Приклад класу та методу для тестування:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace forTest
{
   public class MyNumber
    {

        public MyNumber() { }
        public int Min(int a, int b)
        {
            int res = 0;
            if (a < b) res = 1;
            if (a > b) res = 2;
            return res;
        }
    }
    class Program
```

```
    {
        static void Main(string[] args)
        {
        }
    }
}
```

Зазвичай, тестовий метод складається з частини, що відповідає за ініціалізацію даних необхідних для запуску тесту, з самого тесту і перевірки однієї або декількох умов.

```
using System;
using Microsoft.VisualStudio.TestTools.UnitTesting;
using forTest; /*проект, який тестується*/
namespace UnitTestProject1
{
    [TestClass]
    public class UnitTest1
    {
        [TestMethod]
        public void TestMethod1()
        {
           /*вхідні дані*/
            int a = 1;
            int b = 2;
           /*вихідні дані*/
            int ex_res = 1;
           /*Створення об’єкту класу для подальшого тестування
його методів */
            MyNumber x = new MyNumber();

            /*отриманий результат*/
            int act_res = x.Min(a, b);
            /*порівняння очікуваного та реального результатів*/
            Assert.AreEqual(ex_res, act_res);
        }
        //----------------------------------------

        [TestMethod]
        public void TestMethod2()
        {
            int a = 2;
            int b = 1;
            int ex_res = 2;

            MyNumber x = new MyNumber();

            int act_res = x.Min(a, b);
            Assert.AreEqual(ex_res, act_res);
        }
        //----------------------------------------
        [TestMethod]
        public void TestMethod3()
        {
```

```
            int a = 2;
            int b = 2;
            int ex_res = 0;

            MyNumber x = new MyNumber();

            int act_res = x.Min(a, b);
            Assert.AreEqual(ex_res, act_res);
        }
        //---------------------------------------
        [TestMethod]
        public void TestMethod4()
        {
            int a = -2;
            int b = 2;
            int ex_res = 0;

            MyNumber x = new MyNumber();

            int act_res = x.Min(a, b);
            Assert.AreEqual(ex_res, act_res);
        }
    }
}
```

Для того щоб працювати з модульними тестами відкрийте вікно Test Exlorer-a: Test -> Windows -> Test Explorer. Але після відкриття в ньому тестів не буде, незважаючи на те, що метод вже написали. Для того, щоб побачити тест у Test Explorer-е, побудуйте рішення (рис. 3).

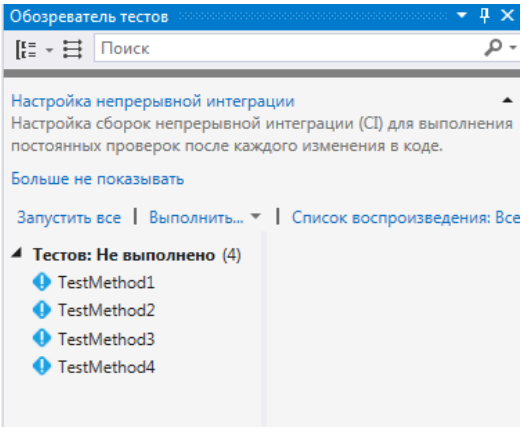


Рис. 3. Завершення побудови тесту

У Test Explorer можна запускати: всі тести, відкривши список, що випадає поруч з Run; невдалі (при виконанні яких очікуваний і отриманий результат не співпали); ті, що не виконувалися; ті, що вже виконано; ті, які були запущено при останньому тестуванні. З контекстного меню на тесті можна запустити конкретний тест, перейти до його опису, налагодити його і т.д. Результати тестування подано нижче (рис. 4).

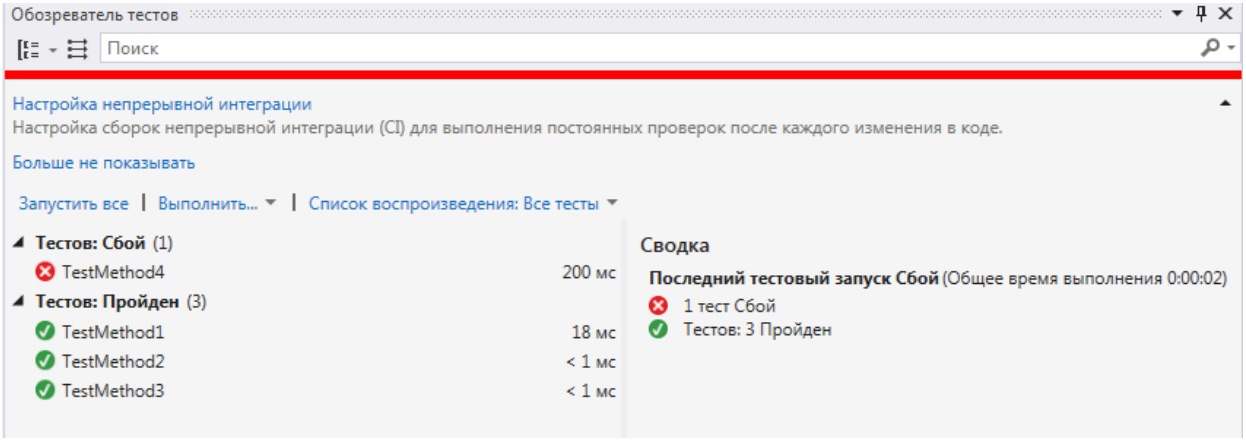


Рис. 4. Результати unit-тестування

Як бачимо, три тести пройдено успішно. Четвертий тест не пройдено, проте це є коректним результатом, оскільки він був направлений на перевірку, чи має функція, що тестується, хибні спрацювання (тут при некоректних даних тесту останній не виконався).

Завдання

Підготувати дві чи більше функцій для тестування. Функції повинні містити не менше шести керуючих конструкцій (розгалужень і циклів). Зазначені структури мають бути вкладеними. Виконати тестування функцій методами чорної скриньки та unit-тестування.

Контрольні питання

1. Методи тестування чорною скринькою.
2. Метод еквівалентного розбиття.
3. Метод аналізу граничних значень.
4. Метод припущення про помилку.
5. Метод функціональних діаграм.
6. Поняття модульного тестування, його мета.
7. Поняття модульного тесту.

Контрольні питання підвищеної складності

1. Відмінності тестування білою та чорною скриньками.
2. Переваги та недоліки тестування чорною скринькою.
3. Стратегія тестування.
4. Тестування багатомодульних програм.
5. Переваги та недоліки unit-тестування порівняно зі звичайним, «ручним» тестуванням.

ЛАБОРАТОРНА РОБОТА № 3

Тема. Проектування структур даних ефективних за показниками обчислювальної складності.

Мета роботи. Вивчити комбінаторно-ймовірнісні методи визначення показників обчислювальної складності алгоритмів та отримати навички засто-

сування цих методів при вирішенні задач розробки ефективних структур даних.

Зміст роботи:

1) виконати підготовку до роботи:
   – ознайомитися з теоретичними відомостями;
   – підготувати зразки даних (див. пояснення у пункті «Завдання»), придатних для подальшого аналізу;
   – вивчити базові методи визначення показників обчислювальної складності алгоритмів;
2) виконати проектування альтернативних структур даних та операцій над ними на абстрактному та логічному рівнях;
3) виконати проектування структур даних на рівні представлення: розробити програму, що формує альтернативні структури даних та забезпечує необхідні операції над ними;
4) розрахувати показники обчислювальної складності операцій обробки структурованих даних;
5) обґрунтований вибір структур даних за допомогою комбінаторно-ймовірнісних методів;
6) оцінити якість виконання практичної частини роботи з представленням результатів у висновках.

Зміст звіту:

1) вид і номер роботи, дисципліна, інформація про виконавця (група, П. І. Б.);
2) тема, мета лабораторної роботи;
3) постановка задачі і матеріали для визначення типового наповнення майбутніх структур;
4) інформацію про структури даних:
   – можливі структури даних та аргументи про доцільність їх застосування;
   – проекти трьох обраних структур даних на абстрактному рівні;
5) критерії вибору мови програмування, що була використана для реалізації обраних структур та їх операцій;
6) текст розробленої програми формування та використання альтернативних структур даних з достатнім рівнем коментованості;
7) розрахунок показників обчислювальної складності обробки структурованих даних, наведений у наступному порядку:
   – визначення випадків кращої, середньої та гіршої обчислювальної складності операцій обробки даних;
   – підрахунок кількості виконаних операторів для операцій обробки даних у вигляді таблиць 8, 9;
   – розрахунок загальної кількості виконаних операторів для операцій обробки даних;

– розрахунок обчислювальної складності операцій обробки даних та їх представлення у вигляді таблиці 11.

8) аналіз результатів з обґрунтуванням вибору структури даних. Обґрунтування вибору структури даних має спиратись на проведені раніше дослідження. Аналіз результатів повинен формувати здатність об'єктивного оцінювання виконаної роботи для виявлення її переваг та недоліків.
9) висновки з оцінкою якості виконаної роботи.

Теоретичні відомості

1. Базові поняття з проектування структур даних.

За визначенням Н. Вірта, програма – це сукупність алгоритмів та структур даних. Визначення відображає важливість проектування структур.

Розрізняють прості та структуровані дані. Прості – це дані доступ до яких можливий як до неподільного об'єкту. Структуровані дані складаються з простих даних (елементів), пов'язаних між собою зв'язками, що визначають порядок розташування та можливу послідовність обробки елементів структури.

Найбільш вживаними серед класичних структур даних є такі, як індексований масив, запис, список, дерево, стек, дек та черга. На даний час поширеними структурами є асоціативний масив, колекція та інші. В мовах програмування такі структури є наперед визначеними, примітивними.

Проектування та розробка структур даних зводиться або до вибору серед наперед визначених структур, або до конструювання більш складних структур. Конструювання полягає у вкладеності одних структур в інші, тобто елементами структур виступають структури.

Конструювання виконують за потребою задачі, яка розв'язується алгоритмічно. Приклади сконструйованих структур: масив масивів, дерево списків, стек записів, колекція дерев масивів та інші.

Ефективність структури даних визначається ефективністю операцій обробки, які реалізовані у вигляді певних алгоритмів та можуть складатись з операцій доступу, зміни порядку та розташування структурованих елементів. Ефективність пов'язана з певними критеріями, такими як час виконання, потреба пам'яті, надійність, складність, тощо. Для об'єктивного визначення критеріїв слід використовувати значення їх показників.

2. Порядок визначення показників обчислювальної складності.

В даній лабораторній роботі ставиться задача вибору ефективної структури даних за критеріями обчислювальної складності. Для визначення показників обчислювальної складності існують такі класичні підходи:

– безпосередній аналіз алгоритму за Д. Кнутом;

– визначення асимптотичної обчислювальної складності;

– оцінка амортизаційної вартості;

– імовірнісний підрахунок операцій.

Кожен з наведених підходів має свої особливості та власну специфіку використання. Основна перевага зазначених підходів – можливість отримання оцінки ще до виконання програми.

Розглянемо використання імовірнісного підрахунку операцій. За цим підходом слід виконати наступну послідовність дій:

– проектування альтернативних структур даних на абстрактному та логічному рівнях;
– проектування структур даних на рівні представлення – реалізація засобами певної мови програмування операцій над обраними структурами;
– аналіз операцій над елементами структур (визначення найкращого, середнього та найгіршого випадків обробки даних);
– розрахунок обчислювальної складності операцій обробки структур даних.

Далі розглянемо кожен з кроків більш детально.

3. Проектування альтернативних структур даних.

Проектування даних виконується у наступній послідовності: на абстрактному, логічному, представлення та фізичному рівнях.

На абстрактному рівні визначаються призначення даних та операції над ними. Окрім цього можуть бути введені певні обмеження, наприклад, максимальна кількість елементів, діапазон значень даних тощо. Зв'язки між даними на абстрактному рівні не визначаються.

Приклад. Структура даних – таблиці ідентифікаторів у лексичному аналізаторі трансляторів. Призначення – зберігання інформації про ідентифікатори, які використовуються у програмі, що транслюється. Операції: додавання та пошуку ідентифікатору, його адреси, типу.

На основі абстрактного виконується логічне проектування. На цьому рівні здійснюється вибір серед примітивних або конструювання на їх основі більш складних структур даних. Для подальшої демонстрації оберемо в якості прикладу дві структури: упорядкований однозв'язний динамічний список та упорядкований статичний масив, а також операції над ними – додавання та пошук елементу.

4. Проектування структур даних на рівні представлення.

Проектування структур даних на рівні представлення полягає у визначенні самих структур даних та їх операцій засобами мови програмування.

Приклад реалізації операцій над елементами упорядкованого однозв'язного динамічного списку на мові С++:

```
int *first = NULL;       //вказівник на перший елемент списку

//реалізація операції додавання нового елементу
void InsertList(int n)
{
    int *node;           //вказівник для нового елементу
    node = (int*)malloc(sizeof(int*)); //виділяється пам'ять під
елемент:
    //половина – під безпосереднє зберігання даних
```

```
    //половина – під зберігання адреси наступного елементу

    //заноситься значення до нового елементу
    node[0] = n;          //у перші 4 байта – нові дані
    node[1] = NULL;       //у інші 4 – NULL-адресу
    //список порожній або дані нового елементу менші ніж дані
першого?
    if(!first || (first[0] > node[0]))
    {
     node[1] = (int)first;   //вставка нового елементу у початок
     first = node;
    }
    else            //список не порожній
    {
     int *tmp = first;        //тимчасовий вказівник

      //пошук місця для вставки нового елементу
     while(tmp[1] && ((int*)tmp[1])[0] <= node[0])
     {
         tmp = (int*)tmp[1];
     }

     node[1] = tmp[1];  //вставка нового елементу перед знайденим
     tmp[1] = (int)node;

   }
}

//реалізація операції пошуку по даним
int* SearchList(int data)
{
  int *tmp = first;       //тимчасовий вказівник для обходу
  int *res = NULL;        //вказівник на результат пошуку
  //пошук серед елементів списку
  while(tmp[1] && tmp[0] <= data)
  {
    if(tmp[0] == data)   //дані співпали?
        res = tmp;  //запам'ятовується результат
    tmp = (int*)tmp[1];  //наступний елемент
  }
  return res;  //повернення результату пошуку
}
```

Приклад реалізації операцій над елементами упорядкованого статичного масиву на мові С++:

```
const int full_size = 1000;   //максимальна  кількість  елементів
масиву
int current_size;             //поточна кількість елементів
int array[full_size];         //статичний масив

//реалізація операції додавання нового елементу
void InsertArray(int data)
{
```

```
  if(current_size + 1 <= full_size)     //масив повний?
                          //(поточна кількість дорівнює максималь-
                          ній?)
  {
    int i, temp;          //тимчасові змінні
    i = current_size;          //лічильник для обходу масиву
    //пошук позиції та звільнення місця для вставки
    while(i > 0 && array[i - 1] > data)
    {
        array[i] = array[i - 1];           //зсув елементів
        i--;
    }
    array[i] = data;          //вставка нових даних
    current_size++;      //збільшення поточної кількості елемен-
тів
 }
}
//операція пошуку
int SearchArray(int data)
{
  int res = -1;          //результат пошуку
  int i = 0;        //лічильник
  //пошук серед елементів масиву
  while(i < current_size && array[i] <= data)
  {
    if(array[i] == data) //дані співпали?
       res = i;               //запам'ятовується  позиція  знайде-
ного елементу
    i++;
  }
  return res;       //повернення результату пошуку
}
```

5. Аналіз операцій над елементами структур даних.

Аналіз операцій полягає у визначенні найкращого, середнього та найгіршого випадків обробки даних у розроблених структурах.

Результат аналізу операції над структурами, що реалізовані у прикладі наведені нижче.

Для **операції додавання:**

- *найкращий випадок* – додавання вузла у початок списку, що передбачає виконання лише першого умовного оператору; для масиву – додавання елементу у кінець, для якого не потрібне виконання циклу зсуву елементів;
- *середній випадок* – додавання вузла у середину списку, для якого потрібно виконати обхід половини вузлів та операцію вставки; для масиву – додавання елементу у середину, яке потребує виконання зсуву половини елементів;
- *найгірший випадок* – додавання у кінець списку, для якого необхідно виконати обхід усіх вузлів; для масиву – додавання елементу у початок, яке потребує виконання зсуву усіх елементів.

Для **операції пошуку** у списку та масиві**:**

– *найкращий випадок* – пошук неіснуючого елементу, меншого за початковий, при якому не потрібно виконувати цикл з перегляду вмісту структури даних;
– *середній випадок* – пошук існуючого елементу, що знаходиться у середині, при якому потрібно виконати перегляд половини вмісту структури даних;
– *найгірший випадок* – пошук неіснуючого елементу, більшого за кінцевий, при якому необхідно здійснити перегляд усього вмісту структури даних.

6. Підрахунок кількості виконаних операторів.

Будемо вважати, що на поточний момент у структурах даних міститься n елементів.

Здійснимо підрахунок кількості виконаних операторів при обробці лінійного списку. Розглянемо приклад, використавши оператор циклу:

```
while(tmp[1] && ((int*)tmp[1])[0] <= node[0])
{
    tmp = (int*)tmp[1];
}
```

У прикладі:
– у найкращому випадку цикл не виконується, оскільки він знаходиться у альтернативній гілці оператору розгалуження, що має істинну умову;
– середній випадок має $n/2$ виконань операторів тіла циклу, оскільки це відповідає середині списку та $n/2+1$ виконання перевірки умови, відповідно до правил роботи циклів з передумовою;
– найгірший випадок має n виконань перевірки умови, тому що на кожному кроці перевіряється наявність наступного вузла та $n-1$ виконань операторів тіла циклу (на останньому кроці наступний вузол відсутній).

Для упорядкованого однозв'язного динамічного списку результат оцінювання операцій наведено у табл. 8.

Виконаємо підрахунок кількості операторів при обробці статичного масиву. Розглянемо підрахунок на прикладі оператору циклу:

```
while(i > 0 && array[i - 1] > data)
 {
     array[i] = array[i - 1];
     i--;
 }
```

Для наведеного оператору:
– у найкращому випадку перевірка умови виконується один раз, для виявлення того, що зсув елементів не потрібен; оператори тіла циклу не виконуються;
– у середньому випадку оператори тіла циклу мають $n/2$ виконань, тому як це відповідає середині масиву; умова перевіряється $n/2+1$ разів, відповідно до правил роботи циклів з передумовою;
– у найгіршому випадку оператори тіла циклу мають n виконань, тому як кожен елемент має бути зсунутий: умова перевіряється $n+1$ разів, відповідно до правил роботи циклів з передумовою.

Таблиця 8

Підрахунок кількості виконаних операторів при обробці лінійного списку

Операції та оператори	Кількість виконаних операторів		
	Найкращий випадок	Середній випадок	Найгірший випадок
Додавання нового елементу			
int *node;	1	1	1
node = (int*)malloc(sizeof(int*));	1	1	1
node[0] = n;	1	1	1
node[1] = NULL;	1	1	1
if(!first || (first[0] > node[0]))	1	1	1
{			
node[1] = (int)first;	1	0	0
first = node;	1	0	0
}			
else	0	1	1
{			
int *tmp = first;	0	1	1
while(tmp[1] && ((int*)tmp[1])[0] <= node[0])	0	n/2 + 1	n
{	0	n/2	n - 1
tmp = (int*)tmp[1];			
}	0	1	1
node[1] = tmp[1];	0	1	1
tmp[1] = (int)node;			
}			
Разом	**7**	**10 + n**	**8 + 2n**
Пошук елементу			
int *tmp = first;	1	1	1
int *res = NULL;	1	1	1
while(tmp[1] && tmp[0] <= data)	1	n/2 + 1	n
{			
if(tmp[0] == data)	0	n/2	n - 1
res = tmp;	0	1	0
tmp = (int*)tmp[1];	0	n/2	n
}			
return res;	1	1	1
Разом	**4**	**5 + 3n/2**	**2 + 3n**

Для статичного масиву результат оцінювання операцій наведено у табл. 9.

Кращий випадок для обох операцій – це константне значення, яке не залежить від кількості елементів у структурах. При середньому та гіршому випадку отримані показники є функціями від n.

7. *Розрахунок обчислювальної складності операцій обробки структур даних.*

Визначимо обчислювальну складність операцій для розроблених струк-

тур за комбінаторно-імовірнісним методом.

Припустимо, що для деякої структури даних реалізовано m операцій. Тоді ймовірність використання будь-якої з операцій дорівнює:

$$P_j = \frac{k_j}{\sum_{i=1}^{m} k_i}, \qquad (1)$$

де P_j – ймовірність виконання j-ої операції обробки даних, k_i – кількість виконань i-ої операції обробки даних. У (1) повинні бути враховані усі реалізовані операції обробки структури даних.

Таблиця 9

Підрахунок кількості виконаних операторів при обробці статичного масиву

Операції та оператори	Кількість виконаних операторів		
	Найкращий випадок	Середній випадок	Найгірший випадок
Додавання нового елементу			
if(current_size + 1 <= full_size)	1	1	1
{			
int i, temp;	1	1	1
i = current_size;	1	1	1
while(i > 0 && array[i - 1] > data)	1	n/2 + 1	n + 1
{			
array[i] = array[i - 1];	0	n/2	n
i--;	0	n/2	n
}			
array[i] = data;	1	1	1
current_size++;	1	1	1
}			
Разом	**6**	**6 + 3n/2**	**6 + 3n**
Пошук елементу			
int res = -1;	1	1	1
int i = 0;	1	1	1
while(i < current_size && array[i] <= data)	1	n/2 + 1	n + 1
{			
if(array[i] == data)	0	n/2	n
res = i;	0	1	0
i++;	0	n/2	n
}			
return res;	1	1	1
Разом	**4**	**5 + 3n/2**	**4 + 3n**

Для прикладу розглянемо задачу формування та використання таблиці ідентифікаторів при розробці деякого транслятору. Під час обробки таких таблиць використовують лише операції додавання та пошуку (в інших структурах даних можуть бути задіяні операції видалення, сортування та інші).

Використаємо отримані значення P_j, для визначення обчислювальної складності операцій за комбінаторно-імовірнісним методом:

$$S = \sum_{j=1}^{m} P_j \cdot k_j , \qquad (2)$$

де S – обчислювальна складність операції обробки структури даних, P_j – ймовірність виконання j-ої операції, k_j – розрахована кількість виконань операторів у j-ій операції обробки структури даних. Для кожної структури даних оцінка S розраховується тричі: для найкращого, середнього та найгіршого випадків відповідно.

Кількість операторів k_j може бути визначена статистичним методом, за допомогою підрахунку ідентифікаторів та відповідних операцій у зразках текстів програм.

Використаємо зразок програми на мові Pascal, наведений у табл. 10.

Під час обробки наведеного тексту лексичним аналізатором операція додавання у таблицю ідентифікаторі буде виконана 17 разів, а операція пошуку – 44. Тоді розрахуємо ймовірність використання операцій згідно з (1):

$$P_{+} = \frac{17}{17+44} = 0.28 \text{ та } P_{?} = \frac{44}{17+44} = 0.72,$$

де P_{+} – ймовірність виконання операції додавання, $P_{?}$ – ймовірність виконання операції пошуку.

Правильність розрахунку перевіряється за умовою $P_{+} + P_{?} = 1$.

Після операцій обробки кількість елементів (n) у структурах даних, які містять лексеми розібраної програми, буде дорівнювати сімнадцяти. Використаємо отримані значення згідно з (2) та визначимо обчислювальну складність операцій зі списком:

$$\hat{S} = 0,28 \cdot 7 + 0,72 \cdot 4 = 4,84$$

$$\bar{S} = 0,28 \cdot (10 + n) + 0,72 \cdot (5 + 3n/2) = 0,3 \cdot 27 + 0,72 \cdot 30,5 = 30,06$$

$$\breve{S} = 0,28 \cdot (8 + 2n) + 0,72 \cdot (2 + 3n) = 0,28 \cdot 42 + 0,72 \cdot 53 = 49,92$$

Знову використаємо (3.2) та визначимо обчислювальну складність операцій з масивом:

$$\hat{S} = 0,28 \cdot 6 + 0,72 \cdot 4 = 4,56$$

$$\bar{S} = 0,28 \cdot (6 + 3n/2) + 0,72 \cdot (5 + 3n/2) = 0,28 \cdot 31,5 + 0,72 \cdot 30,5 = 30,78$$

$$\breve{S} = 0,28 \cdot (6 + 3n) + 0,72 \cdot (4 + 3n) = 0,28 \cdot 57 + 0,72 \cdot 55 = 55,56$$

Таблиця 10

Зразок програми для аналізу та підрахунку кількості використань операторів обробки структур даних

Текст програми	Кількість використань	
	Операція додавання	Операція пошуку
procedure initRound(round:integer);		
var		
i,j: byte;	2	
isChosen: boolean;	1	
a,b,c,d:word;	4	
selectedPin: integer;	1	
attempt, round, wins, field: integer;	4	
winner: boolean;	1	
pins:array[0..maxAttempts-1,0..3] of integer;	1	1
answer: array[0..maxAttempts-1,0..3] of integer;	1	1
passwordPin: array[0..3] of integer;	1	
passwordHelp: array[0..1,0..3] of boolean;	1	
begin		
randomize;		
for i := 0 to maxAttempts-1 do begin		2
for j:=0 to 3 do begin		1
pins[i][j] := -1;		3
answer[i][j] := -1;		3
end;		
end;		
for i := 0 to 3 do begin		1
isChosen := true;		1
while (isChosen) do begin		1
isChosen := false;		1
gettime(a,b,c,d);		4
passwordPin[i] := (random(a*60+d)) mod 6;		4
for j:=0 to i do		2
if (i<>j) and (passwordPin[j] = passwordPin[i])		6
then		
isChosen := true;		1
end;		
end;		
for i := 0 to 3 do begin		1
pins[attempt][i] := -1;		3
end;		
nrButtonActive := false;		1
acceptButtonActive := false;		1
roundEnd := false;		1
round := round;		2
attempt := 0;		1
field := -1;		1
selectedPin := -1;		1
end;		
Разом	**17**	**44**

Підсумкові результати розрахунку представимо у вигляді табл. 11.

Таблиця 11

Результати розрахунку обчислювальної складності

Структура даних	Обчислювальна складність		
	Найкращий випадок	Середній випадок	Найгірший випадок
Упорядкований однозв'язний список	4,84	30,6	49,92
Упорядкований статичний масив	4,56	30,78	55,56

Виконані дослідження дають змогу обрати кращу структуру за критерієм обчислювальної складності. У даному випадку кращою структурою для представлення таблиці ідентифікаторів є упорядкований однозв'язний список.

Теоретичні відомості для поглибленого вивчення

1. *Доцільність визначення показників обчислювальної складності.*

Розглянемо у якості прикладу структури даних довідник, інформація у якому відсортована за алфавітом. Операція пошуку може мати декілька реалізацій: пошук перебором, бінарний пошук та інші. Для пошуку перебором можлива кількість повторень може складати від 1 до n, де n – кількість сторінок у довіднику, у середньому $n/2$. Бінарний пошук полягає у розділенні області пошуку навпіл. Та частина, що містить потрібні дані знову ділиться навпіл. У середньому кількість повторень буде складати $\log_2 n$.

Порівняємо операції пошуку: пошук перебором при n = 10 матиме у середньому 5 повторень, бінарний – 3...4, при n = 128, показники алгоритмів дорівнюватимуть 64 та 6, відповідно. При зростанні n, різниця стає більш вираженою, але реалізація другого варіанту складніша, тому для вибору потрібна коректна оцінка обох операцій.

Застосування показників обчислювальної складності є доцільним якщо:

- передбачається обробка значного об'єму даних;
- необхідно задовольнити підвищені показники до часу обробки;
- відсутнє розпаралелювання обчислень програмними та апаратними засобами.

2. *Методи та підходи до оцінки операцій обробки структур даних.*

Як було показано раніше кожною операцією обробки структур даних є процедура, функція або частина програми, розроблена за певним алгоритмом. Тому ключову роль у ефективності структур даних відіграють часові характеристики алгоритмів. Розглянемо декілька підходів до визначення часових показників. Їх основою є:

- безпосередній аналіз алгоритму;
- імовірнісний підрахунок операцій;
- асимптотична обчислювальна складність;
- оцінка амортизаційної вартості.

Кожен з наведених класичних підходів має свої особливості та власну специфіку використання. Їх основна перевага у відсутності необхідності виконання програми.

3. Безпосередній аналіз алгоритму.

Застосування підходу базується на досвіді оцінки типових алгоритмів. Наприклад, група операторів знаходиться у тілі циклу, виконання якого не переривається. На основі досвіду можна припустити, що складність такого циклу дорівнює n, де n – кількість виконань операторів тіла циклу. Якщо тіло циклу – цикл, кількість виконань якого m, складність становитиме – $n*m$ (або n^2 у випадку, коли $n=m$).

Розглянемо, як приклад, пошук мінімального елементу матриці:

```
for(i = 0; i < n; i++)
{
  for(j = 0; j < n; j++)
  {
    if(matrix[i][j] < min)
    {
      min = matrix[i][j];
    }
  }
}
```

Згідно з умовою, оператори тіла зовнішнього циклу виконаються n разів. Внутрішній цикл також виконається n разів для кожного виконання зовнішнього циклу. Відповідно, перевірка умови, яка належить внутрішньому циклу, виконається n^2 разів.

Недоліком підходу є низька придатність для великих алгоритмів, та алгоритмів, виконання яких може змінюватись в залежності від комбінації вхідних даних.

4. Імовірнісний підрахунок операцій.

Обчислювальна складність, запропонована Д. Кнутом, у методі підраховується за допомогою кількості операцій, вважаючи, що час їх виконання однаковий. З урахуванням часу виконання операцій (3.2) перетворюється на

$$S = \sum_{j=1}^{m} P_j \cdot k_j \cdot t_j ,$$

де t_j – час виконання j-ої операції.

Якщо не враховувати розпаралелювання на конвеєрі процесора та інші методи прискорення обчислень, можна встановити час виконання операцій за латентністю команд процесору. Наприклад, у табл. 12 наведений асемблерний код, отриманий з вікна відлагодження у середовищі програмування.

Таблиця 12

Підрахунок кількості операцій з урахуванням латентності команд процесору

Оператори мови С++	Адреса у ОП	Команди процесору	Латентність[1]
int* SearchList(int data)	003914E0	push ebp	1.5
	003914E1	mov ebp,esp	0.5
	003914E3	sub esp,0D8h	1
	003914E9	push ebx	1.5
	003914EA	push esi	1.5
	003914EB	push edi	1.5
	003914EC	lea edi,[ebp-0D8h]	3
	003914F2	mov ecx,36h	0.5
	003914F7	mov eax,0CCCCCCCCh	0.5
	003914FC	rep stos dword ptr es:[edi]	15+36
	003914FE	mov eax,dword ptr [first	0.5
	00391503	(3980DCh)]	0.5
	00391506	mov dword ptr [tmp],eax	0.5
	0039150D	mov dword ptr [res],0	0.5
	00391511	cmp dword ptr [tmp],0	2
	00391513	je SearchList+58h (391538h)	0.5
	00391516	mov eax,dword ptr [tmp]	0.5
	00391518	mov ecx,dword ptr [eax]	0.5
	0039151B	cmp ecx,dword ptr [data]	2
	0039151D	jg SearchList+58h (391538h)	0.5
	00391520	mov eax,dword ptr [tmp]	0.5
	00391522	mov ecx,dword ptr [eax]	0.5
	00391525	cmp ecx,dword ptr [data]	2
	00391527	jne SearchList+4Dh (39152Dh)	0.5
	0039152A	mov eax,dword ptr [tmp]	0.5
	0039152D	mov dword ptr [res],eax	0.5
	00391530	mov eax,dword ptr [tmp]	0.5
	00391533	mov ecx,dword ptr [eax+4]	0.5
	00391536	mov dword ptr [tmp],ecx	2
	00391538	jmp SearchList+2Dh (39150Dh)	0.5
	0039153B	mov eax,dword ptr [res]	1.5
	0039153C	pop edi	1.5
	0039153D	pop esi	1.5
	0039153E	pop ebx	0.5
	00391540	mov esp,ebp	1.5
	00391541	pop ebp	8
		ret	
Разом			**93**

5. Асимптотична обчислювальна складність.

Досить часто для визначення переваги одного алгоритму відносно іншого не потрібно знати точну оцінку їх складності. Достатньо оцінити асимптоти-

[1] Латентність – кількість тактів процесара від входу до виходу команди з конвеєру.

ку зростання часу виконання наданих алгоритмів. Якщо швидкість зростання часу виконання одного алгоритму при збільшенні кількості вхідних даних нижча за швидкість зростання часу іншого алгоритму, перший алгоритм є більш ефективним.

Розглянемо загальний принцип асимптотичних оцінок. Нехай є деякі функції $g(n)$ та $f(n)$. Якщо існують такі значення c_1, c_2 та n, і починаючи з деякого n_0 виконується умова $c_1 g(n) \le f(n) \le c_2 g(n)$, при $c_1 > 0$, $c_2 > 0$ та $n > n_0$, прийнято говорити, що функція $f(n)$ обмежена функцією $g(n)$. У вигляді асимптотичних позначок ситуація має наступний вигляд:

$$f(n) = \Theta(g(n)). \quad (3)$$

Запис (3) містить у собі дві оцінки: верхню та нижню, тому, при необхідності, їх можна розділити. Так, наприклад, якщо починаючи з n_0 виконується умова $f(n) \le cg(n)$, при $c > 0$ та $n > n_0$, прийнято говорити, що функція $f(n)$ обмежена функцією $g(n)$ зверху. А якщо виконується умова $cg(n) \le f(n)$ – обмежена знизу. Першу оцінку прийнято позначати, як O, другу – Ω, та записувати відповідно:

$$f(n) = O(g(n)), \quad (4)$$

$$f(n) = \Omega(g(n)). \quad (5)$$

Нижче наведено графічне представлення розглянутих ситуацій:

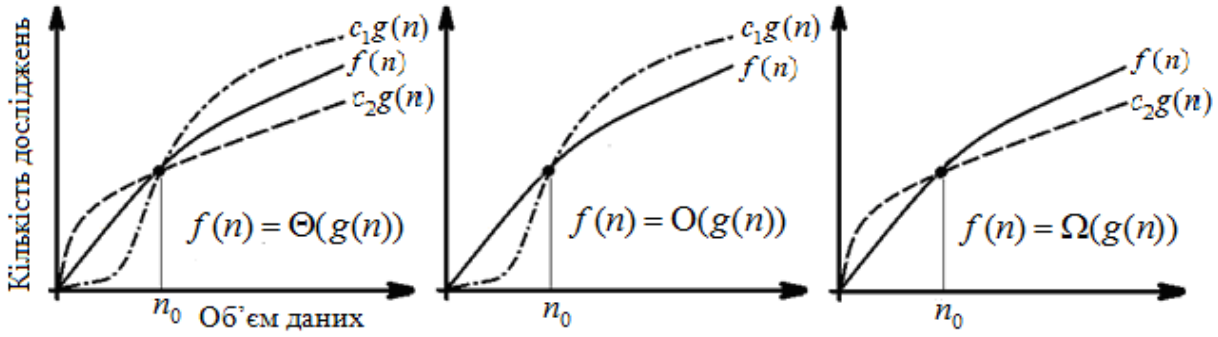


Рис. 5. Графічне представлення асимптотичних обмежень

Задача визначення асимптотичної обчислювальної складності може бути вирішена різними методами. Розглянемо ті, що базуються на рекурсії. Оцінка кількості обчислень алгоритмом, у таких методах, може бути представлена як *рекурентне співвідношення*. Співвідношення такого типу зв'язує оцінку для однієї кількості вхідних даних з оцінкою для меншої кількості.

Розглянемо реалізацію рекурсивного алгоритму пошуку місця розташування (індексу) деякого елементу масиву.

```
1   int BinarySearchRE(int *data, int left, int right, int key)
    {
2     int center = (left + right)/2;
3     int result;
4     if(right - left <= 0)
      {
```

```
5       result = -1;
      }
6     if(data[center] < key)
      {
7       result = BinarySearchRE(data, center, right, key);
      }
8     if(key < data[center])
      {
9       result = BinarySearchRE(data, left, center, key);
      }
10    if(key == data[center])
      {
11      result = center;
      }
12    return result;
    }
```

Нехай n – кількість елементів масиву, $T(n)$ – загальна кількість обчислень для пошуку елементу. Вона обчислюється за допомогою двох доданків: $T(n/2)$ та f. Тут f – кількість обчислень у операторах 1...6, 8, 10 та 12, $T(n/2)$ – кількість виконань одного з операторів 7 або 9 для наполовину меншою кількості елементів масиву. Тобто

$$T(n) = T(n/2) + f. \tag{6}$$

У більш загальному випадку кількість обчислень деяких рекурсивних алгоритмів

$$T(n) = aT(n/b) + f(n), \tag{7}$$

де $a \geq 1$ і $b > 1$ – деякі коефіцієнти. У розглянутому вище окремому випадку (3.6) $a = 1$, $b = 2$, f – константа.

Не слід вважати, що рекурентне співвідношення (6) стосується тільки рекурсивних алгоритмів. Той же алгоритм пошуку може бути реалізовано ітераційно, за допомогою циклу.

```
1   int BinarySearchLE(int *data, int size, int key)
    {
2     int left = 0, right = size;
3     int center, result = -1;
4     do
5        center = (left + right)/2;
6        if(data[center] < key)
         {
7          left = center;
         }
8        if(key < data[center])
         {
9          right = center;
         }
10    }while(right - left > 0 && key != data[center]);
11    if(key == data[center])
      {
12      result = center;
      }
13    return result;
    }
```

Загальна кількість обчислень $T(n)$ у наведеній реалізації бінарного пошуку обчислюється, як (6). Тут $T(n/2)$ – кількість обчислень у операторах тіла циклу 4...10 для половини елементів масиву та f – оператори 1...3 та 11...13.

Наведені приклади демонструють яким чином отримується співвідношення типу (7).

Розглянемо методи, які дозволяють визначити відповідну асимптотичну оцінку, знаючи рекурентне співвідношення:

– метод підстановки;

– метод ітерацій;

– загальний метод (за доведеною теоремою).

Тобто ці методи дозволяють знайти співвідношення (3), (4) або (5) наперед знаючи, що кількість обчислень відповідає (7).

Метод підстановки або *індуктивний метод* полягає у наступному: спочатку потрібно припустити можливу асимптотичну оцінку. Потім припущення доводиться за допомогою математичної індукції. Припущення робиться на основі досвіду, по аналогії з подібними алгоритмами. Інакше – методом підбору, з поступовим уточненням оцінки.

Основна задача методу математичної індукції – довести істинність предикату (логічного виразу) $P(n)$. Зпочатку доводиться істинність базису індукції $P(1)$. Потім, вважаючи істинним $P(n)$, доводиться істинність $P(n+1)$, тим самим доводиться істинність будь-якого $P(n)$.

Використаємо метод підстановки для прикладу (6). Припустимо, що $T(n)$ має таку асимптотичну оцінку: $O(\log_2 n)$, тобто $T(n) \le c\log_2 n$ при певних значеннях c та $n \ge n_0$. Перевіримо припущення за допомогою індукції.

У наведеному випадку предикатом є $P(n) \equiv T(n) \le c\log_2 n$. Базис індукції $P(1) \equiv T(1) \le c\log_2 1$ є хибним логічним виразом, тому як $\log_2 1 = 0$. Оберемо коефіцієнт c таким, що при $n \in [2,4]$ $P(n)$ є істинним. Базис індукції – $P(t)$, де $t = 2...4$.

З припущенням, що $P(n/2)$ є істинним, доведемо істинність $P(n)$. Предикат $P(n/2) \equiv T(n/2) \le c\log_2(n/2)$ є істинним. Виконаємо підстановку предикату у співвідношення (6):

$$T(n) \le c\log_2(n/2) + f,$$

$$c\log_2(n/2) + f = c\log_2 n - c\log_2 2 + f = c\log_2 n - c + f,$$

$$T(n) \le c\log_2 n, \qquad (8)$$

з обраним $c > f$.

Тоді для будь-якого n, $P(n)$ – істина, тому що існує послідовність n, $\frac{n}{2}$, $\frac{n}{4}$, ... $\frac{n}{q}$, така що $2 \le \frac{n}{q} \le 4$, при якому істинний базис індукції $P(n/q)$.

Розглянутий метод може використовуватись як для нижніх так і для верхніх оцінок.

Якщо припустити оцінку не вдається, використовують *метод ітерацій*. Зміст методу наступний: співвідношення підставляється саме в себе певну кількість разів, достатню для того, щоб представити співвідношення у вигляді ряду або функції.

Розглянемо на прикладі співвідношення (6)

$$T(n) = T(n/2) + f = (T(n/4) + f) + f = T(n/4) + 2f,$$

де f – коефіцієнт не залежний від n, тому f накопичується і не ділиться на степінь двійки. Продовжимо підстановки

$$T(n/4) + 2f = T(n/8) + 3f = T(n/16) + 4f = ...T(n/2^w) + wf.$$

Помітно, що співвідношення змінюється згідно певної тенденції і може бути представлено у загальній формі: $T(n/2^w) + wf$. У даному випадку i – такий коефіцієнт, при якому виконується остання ітерація підстановки. Остання ітерація може бути описана умовою $T(n/2^w) = T(1)$, тобто подальше розділення не можливе. Слід також зазначити, що $T(1) = 0$, адже, якщо елемент в структурі лише однин, він уже є результатом пошуку. Таким чином, можна зробити наступний висновок:

$$T(n) = wf. \qquad (9)$$

Тепер визначимо, яке значення має коефіцієнт i. Спираючись на умову $T(n/2^w) = T(1)$, можна зробити висновок, що $2^w = n$ і відповідно $w = \log_2 n$. Тобто, виконавши підстановку у (9), отримаємо

$$T(n) = f \log_2 n. \qquad (10)$$

Рівняння (10), співпадає з рівнянням (8), яке отримане за допомогою індуктивного методу.

Загальний метод використовується для рекурентних відношень, що мають вигляд (7). Існує наступна теорема.

Нехай $a \geq 1$та $b > 1$ – константи, $f(n)$ – функція, $T(n)$ визначена при додатних значеннях n формулою $T(n) = aT(n/b) + f(n)$, тоді:

1) якщо $f(n) = O(n^{\log_b a - \varepsilon})$ для деякої $\varepsilon > 0$, тоді $T(n) = \Theta(n^{\log_b a})$;

2) якщо $f(n) = \Theta(n^{\log_b a})$, тоді $T(n) = \Theta(n^{\log_b a} \lg n)$;

3) якщо $f(n) = \Omega(n^{\log_b a - \varepsilon})$ для деякої $\varepsilon > 0$ і $af(n/b) \leq cf(n)$ для деякої $c < 0$ та досить великих значень n, тоді $T(n) = \Theta(f(n))$.

Розглянемо у якості прикладу вже знайоме співвідношення (6):

$$T(n) = T(n/2) + f,$$

де $a = 1$, $b = 2$, $f(n) = t$. Для даного співвідношення $f(n) = \Theta(1) = \Theta(n^0) = \Theta(n^{\log_2 1}) = \Theta(n^{\log_b a})$, тобто виконується умова 2 теореми. Тоді

$$T(n) = \Theta(\lg n). \qquad (11)$$

6. Оцінка амортизаційної вартості.

Амортизаційний аналіз використовується для оцінки часу виконання послідовності операцій над певною структурою даних. Для визначення верхньої межі оцінки достатньо помножити максимальну тривалість однієї операції на загальну кількість операцій у послідовності. Якщо операції з великим та ма-

лим часом виконання чергуються, оцінка може бути уточнена – вона відповідає середньому часу виконання. Оцінки такого роду називаються амортизаційними.

Для виконання амортизаційного аналізу потрібно визначити певну облікову вартість кожної операції. Облікова вартість може бути більшою або меншою за реальний час виконання операції, але такою, що: для будь-якої послідовності операцій фактична сумарна тривалість їх виконання не повинна перевищувати суму їх облікових вартостей. Якщо умова виконується – облікова вартість коректна.

Для визначення облікової вартості використовуються такі основні методи:

– групування;

– передплати;

– потенціалів.

Метод групування полягає у наступному: для кожного n виконується оцінка часу $T(n)$ у найгіршому випадку, де n – кількість операцій. Час виконання однієї операції оцінюється як $T(n)/n$ та оголошується обліковою вартістю. Облікова вартість усіх операцій вважатиметься однаковою.

Розглянемо в якості прикладу таку структуру даних, як стек. Нехай стек, окрім стандартних операцій додавання та вилучення, має також операцію множинного вилучення. Така операція може бути реалізована, як циклічний виклик стандартного вилучення.

Нехай стандартні операції виконуються за час t, тоді виконання n операцій потребує час рівний nt. Тепер проаналізуємо операцію множинного вилучення. При використанні такої операції для вилучення k елементів зі стеку розміром n, буде викликано k операцій стандартного вилучення, якщо $k \le n$, або n операцій, якщо $k > n$. Тобто при множинному вилученні виконується $\min(k,n)$ операцій. Відповідно, $\min(k,n)$ – це оцінка множинного вилучення.

Якщо у початковому стані стек порожній, кількість усіх операцій стандартного вилучення (у тому числі тих, що входять у множинне вилучення) не перевищує кількість виконаних операцій додавання. Кількість операцій додавання не перевищує n. Висновок: вартість будь-якої послідовності з операцій додавання, вилучення та множинного вилучення, є $T(n) \le tn$, тобто має оцінку $O(n)$. Таким чином кожна з операцій матиме облікову вартість: $O(n)/n = O(1)$.

Метод передплати має такий зміст: кожна операція отримує свою облікову вартість, причому, отримані вартості можуть бути, як більші, так і менші за реальні. Якщо облікова вартість перевищує реальну, різниця розглядається, як резерв. Резерв зв’язується з іншою операцією над структурою даних і вважається передплатою за наступне її виконання. Таким чином, за рахунок резерву компенсується різниця між обліковою та реальною вартостями для операцій, чия облікова вартість нижча за реальну.

Облікові вартості повинні обиратися так, щоб резерв залишався додатним у будь-який момент роботи. Для цього фактична вартість не повинна перевищувати суму облікових вартостей.

У якості прикладу знову розглянемо стек. Визначимо фактичну вартість операцій наступним чином:

– додавання елементу до стеку – 1;

– вилучення елементу зі стеку – 1;

– множинне вилучення – $min(k,n)$.

Призначимо вказаним операціям наступні облікові вартості:

– додавання – 2;

– вилучення – 0;

– множинне вилучення – 0.

У такому випадку кожна операція додавання елементу призводить до накопичення резерву. Операція видалення резерв використовує. Якщо у стек було додано n елементів, сформується передплата розміром n, яка покриватиме виконання операції видалення елементів у будь-якій кількості, що не перевищує n. При цьому не має значення яким чином вилучаються елементи – поодинці або множинним вилученням.

Метод потенціалів можна охарактеризувати, як окремий випадок методу передоплати. Різниця полягає у тому, що резерв не розподілений між окремими операціями, а накопичується в цілому для структури даних. Такий резерв називається «потенціалом» або «потенційною енергією» структури.

Нехай D_i – стан структури після виконання i-ої операції (D_0 – початковий стан), c_i – реальна вартість операції, а Φ – потенціальна функція, яка повинна представити множину можливих станів структури даних у вигляді дійсних чисел. У такому випадку облікову вартість $\tilde{c}_i$ можна виразити за допомогою виразу

$$\tilde{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}). \qquad (12)$$

Вираз (12) дозволяє представити облікову вартість i-ої операції, як суму реальної вартості та приросту потенціалу у результаті її виконання. Також помітно, що за умови додатної різниці потенціалів $\Phi(D_i) - \Phi(D_{i-1}) > 0$ облікова вартість i-ої операції включатиме в себе передплату за наступні операції, у іншому випадку ($\Phi(D_i) - \Phi(D_{i-1}) < 0$) – облікова вартість менше за фактичну і різниця покривається за рахунок вже накопиченого потенціалу.

На основі (12) сумарний потенціал може бути представлений як

$$\sum_{i=1}^{n} \tilde{c}_i = \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0) \qquad (13)$$

Якщо є можливість обрати функцію Φ, так що $\Phi(D_n) \geq \Phi(D_0)$, сумарна облікова вартість даватиме верхню оцінку для реальної вартості послідовності з n операцій. У випадках, коли початковий стан співпадає з порожньою структурою, вважається, що $\Phi(D_0) = 0$.

Розглянемо стек. У якості потенціальної функції приймемо $\Phi = n$. Оскільки у початковому стані стек порожній, маємо: $\Phi(D_n) \geq \Phi(D_0) \Rightarrow \Phi(D_i) \geq 0$, і, відповідно, сума облікових вартостей даватиме оцінку зверху для реальної вартості послідовності з операцій додавання та вилучення.

Проаналізуємо операції стека для визначення облікових вартостей. Встановимо $\Phi(D_0)=0$. Функція стандартного додавання має одиничну реальну вартість і на один підвищує потенціал стеку, отже її облікова вартість, згідно з (12), дорівнює двом. Функція вилучення має одиничну реальну вартість і на один зменшує потенціал. У такому випадку облікова вартість складатиме нуль. Множинне вилучення представляє з себе циклічний виклик вилучення стандартного, тому його облікова вартість також складатиме нуль.

Завдання

Розробити об'єктно-орієнтовану програму, що реалізує три структури даних, визначивши для них операції додавання, видалення, пошуку елементів.

Для тексту програми на заданій мові програмування виконати пошук усіх лексем класу, визначеного варіантом. Знайдені лексеми занести в розроблені структури, виключаючи повтор елементів.

Виконати ймовірнісно-комбінаторну оцінку функцій для роботи зі структурами, на основі якої зробити висновок про оптимальність структури з точки зору найменшої обчислювальної складності.

Класи лексем відповідно варіанту:

1) ідентифікатори;
2) зарезервовані слова;
3) операції;
4) роздільники;
5) коментарі;
6) літерали.

Мову програмування, на якій буде представлено текст програми для лексичного розбору, обирає студент самостійно за погодженням з викладачем.

Контрольні питання

1. Поняття ефективності структури даних.
2. Порядок вибору альтернативних структур даних.
3. Врахування операторів керуючих структур (циклу, розгалуження) при визначенні обчислювальної складності. Визначення, як позначаються оператори такого виду на складності.
4. Підходи до визначення обчислювальної складності.
5. Зв'язок між обчислювальною складністю алгоритмів та обчислювальною складністю структур даних.
6. Розрахунок обчислювальної складності операцій обробки структур даних.
7. Розрахунок обчислювальної складності структур даних.
8. Визначення ймовірності використання операцій над елементами розробленої структури даних.
9. Визначення обчислювальної складності алгоритмів за Д. Кнутом.

Контрольні питання підвищеної складності

1. Вибір структур даних на основі показників обчислювальної складності.
2. Безпосередній аналіз алгоритму.

3. Асимптотична обчислювальна складність алгоритмів.
4. Методи визначення асимптотичної обчислювальної складності програм та алгоритмів.
5. Поняття рекурентного відношення.
6. Метод підстановки.
7. Метод ітерацій.
8. Загальний метод. Теорема про рекурентні відношення.
9. Амортизаційна вартість операцій обробки структур даних.
10. Методи визначення амортизаційної вартості операцій обробки структур даних.
11. Метод групування.
12. Метод передплати.
13. Метод потенціалів.
14. Визначення обчислювальної складності алгоритмів імовірнісними методами.
15. Переваги та недоліки підходів з визначення обчислювальної складності структур даних.
16. Переваги та недоліки методів визначення амортизаційної вартості операцій обробки структур даних.

ЛАБОРАТОРНА РОБОТА № 4

Тема. Проектування структур даних з урахуванням програмно-апаратних середовищ їх використання.

Мета роботи. Вивчити статистичні методи визначення показників часової ефективності алгоритмів та набути практичних навичок застосування цих методів при вирішенні задач розробки структур даних.

Зміст роботи:

1) вивчення методів визначення показників часової ефективності алгоритмів;
2) вивчення методів вибору структур даних та алгоритмів їх обробки з урахуванням показників часової ефективності у визначених програмно-апаратних середовищах;
3) розробка програми для виміру часової ефективності операцій обробки структур даних;
4) підготовка та виконання комп’ютерних експериментів щодо формування та використання альтернативних структур даних (за підготовленими в попередній роботі алгоритмами обробки даних).
5) розробка програми, що визначає часову ефективність окремих алгоритмів обробки даних та алгоритмів обробки даних у сукупності.
6) обґрунтований вибір структур даних для лексичного аналізатору з урахуванням показників часовою ефективністю.

Зміст звіту:

1) вид і номер роботи, дисципліна, інформація про виконавця (група, П. І. Б.);
2) тема, мета лабораторної роботи;
3) постановка задачі з визначеними кращими структурами даних за комбінаторно-імовірнісних методами (попередня лабораторна робота).
4) текст програми, що реалізує альтернативні структури даних, які досліджуються, та операції над даними структурами;
5) опис методів дослідження (з приведеними програмами дослідження).
6) результати комп'ютерних експериментів (часові значення) та визначені показники часової ефективності.
7) аналіз результатів та висновки (обґрунтування вибору однієї з альтернативних структур даних).

Теоретичні відомості

1. Базові поняття з проектування структур даних.

Використання таких показників, отриманих за допомогою комбінаторно-імовірнісного методу, для порівняння альтернативних структур даних (СД) може бути некоректним у програмно-апаратних середовищах із засобами прискорення обчислень. Більш точним інструментом порівняння та вибору є експериментальне визначення показників часової ефективності алгоритмів обробки альтернативних СД.

Для порівняння показників часової ефективності двох структур даних необхідно встановити:

- середню перевагу однієї структури даних над іншою;
- співвідношення областей переваги кожної зі структур даних.

Визначення показників часової ефективності проводиться на основі виконання комп'ютерного експерименту. Експеримент полягає у послідовності випробувань структури даних. Кожне випробування складається з моделювання наповненості структури даних конкретними значеннями та конкретного обсягу, виконанні операції доступу або послідовності операції доступу з заміром часу виконання таких операцій.

Розрахунок показника S – середньої переваги однієї структури даних над іншою, виконується наступним чином

$$S = \frac{1}{N}\sum_{i=1}^{N}\frac{t_i'' - t_i'}{\max(t_i', t_i'')}\cdot 100\%, \tag{14}$$

де t_i', t_i'' – час виконання операції обробки для кожної зі структур у і-му випробуванні, N – кількість випробувань.

Показник R – співвідношення областей переваги може бути представлений як

$$R = \frac{1}{N}\sum_{i=1}^{N} sign(t_i'' - t_i')\cdot 100\%, \;\; sign(a) = \begin{cases} 1, \text{ якщо } a > 0 \\ 0, \text{ якщо } a \le 0 \end{cases}. \tag{15}$$

Виконання серії випробувань підвищує статистичну точність результату, тому як отримані показники наближається до свого середнього значення.

2. Визначення показників числової ефективності.

Для експериментальної оцінки показників часової ефективності алгоритмів обробки структур даних потрібно:
– обрати структури даних для порівняння;
– підготовити комп'ютерні експерименти;
– виконати експерименти;
– розрахувати показники часової ефективності;
– інтерпретувати результати.

Розглянемо як приклад реалізовані у попередній лабораторній роботі структури даних: упорядкований однозв'язний динамічний список та упорядкований статичний масив.

При підготовці експерименту слід підготувати програмні засоби, що реалізують:
– операції доступу до даних;
– засоби виміру часу;
– експеримент і накопичують дані.

Вимірювання часу

Для вимірювання часу виконання операції може використовуватися високоточний системний таймер – QueryPerformanceTimer. Використання таймеру передбачає:
– отримання частоти таймеру frequency;
– отримання стартового значення таймеру time_start;
– отримання кінцевого значення таймеру time_finish;
– розрахунок часу, що був отриманий.

Всі змінні повинні мати тип LARGE_INTEGER.

Нижче наведено приклад використання таймеру для операцій додавання елементу у список та масив:

```
QueryPerformanceFrequency(&frequency);//отримання частоти таймеру
QueryPerformanceCounter(&time_start); //отримання стартового часу
list.Insert(data[tmp_loading]);      //операція додавання у спи-
сок
Sleep(10);
QueryPerformanceCounter(&time_finish); //отримання кінцевого часу
//розрахунок: різниця кінцевого та стартового значень ділиться на
частоту
double    list_time_i    =    (double)(time_finish.QuadPart    -
time_start.QuadPart)                                          /
                    (double)frequency.QuadPart;
```

Аналогічно вимірюється час виконання операції додавання до масиву array_time_i.

Зверніть увагу, що у наведеному прикладі застосована процедура Sleep. Вона призначена для того щоб операції процедури QueryPerfomanceCounter

не випереджали на процесорному конвеєрі виконання нашої процедури додавання.

Для проведення експерименту потрібно реалізувати програму, що виконує випробування.

Визначимо такі зміні, як count_of_data – розмір множини вхідних даних, total_repeat – кількість випробувань (відповідає N), tmp_loading – випадкова кількість вхідних даних для занесення у структури. Вважатимемо, що total_repeat дорівнює 500, count_of_data – 1000.

Організуємо циклічне виконання випробувань. Повторення має відбутися total_repeat разів. Кожного разу обиратимемо tmp_loading – випадкове значення наповненості структур, що досліджуються. Значення tmp_loading повинно лежати в інтервалі від 0 до (count_of_data – 1). Обравши таке значення наповнимо структури відповідною кількістю елементів. Реалізація розглянутих дій може бути такою:

```
for(int i = 0; i < total_repeat; i++)   //циклічне повторення ек-
сперименту
{
   List list;
   Array array;
   tmp_loading = rand()%(count_of_data - 1);          //випадкова
наповненість
   for(int j = 0; j < tmp_loading; j++)          //наповнення
структури
   {
     int data_to_add = rand()%100
     list.Insert(data_to_add);
     array.Insert(data_to_add);
   }
}
```

Виконання експерименту.

Для виконання експерименту слід забезпечити умови, що наближені до реальних, тих, в яких будуть використовуватись спроектовані структури даних. Потрібно звернути увагу на:

– потоки та процеси;

– операційну систему;

– антивірусне програмне забезпечення;

– характеристики ЕОМ;

– характер та об’єми даних, якими будуть наповнені структури.

Розрахунок показників часової ефективності.

Для розрахунку показників обчислювальної складності за (14) та (15), після виконання випробування накопичуємо sup_i, sup_s та reg_i, reg_s:

```
//накопичення отриманих оцінок для операцій над структурами
//накопичення оцінки S для додавання елементу
sup_i += sup(list_time_i, array_time_i);

//накопичення оцінки R для додавання елементу
reg_i += reg(list_time_i, array_time_i);
```

```
//накопичення оцінки S для пошуку елементу
sup_s += sup(list_time_s, array_time_s);
//накопичення оцінки R для пошуку елементу
reg_s += reg(list_time_s, array_time_s);
```

Тут задіяні функції sup та reg:

```
   double sup(double i, double j)
{
   double sup;
   if(i > j)
      sup = (j - i)/i;
   else
      sup = (j - i)/j;
   return sup;
}
```

```
double reg(double i, double j)
{
   double reg;
   reg = j - i;
   if(reg > 0)
      return 1;
   return 0;
}
```

По завершенню експерименту обчислюємо S- та R- оцінки:

```
sup_i = (sup_i/total_repeat)*100; //оцінка S для додавання елементу
sup_s = (sup_s/total_repeat)*100; //оцінка S для пошуку елементу
reg_i = (reg_i/total_repeat)*100; //оцінка R для додавання елементу
reg_s = (reg_s/total_repeat)*100; //оцінка R для пошуку елементу
```

Інтерпретування результатів.

Результатом виконання експерименту є таблиця подібна до табл. 13. Зазначимо, що отримані оцінки можуть відрізнятись від наведених.

Таблиця 13

Результати розрахунку показників обчислювальної складності

Операція	Показники обчислювальної складності	
	Оцінка S-	Оцінка R-
Додавання до структури даних	2,6%	85%
Пошук у структурі даних	-4.5%	98%

Оцінка S- демонструє перевагу списку над масивом. Якщо отримана оцінка додатна (операція додавання) – це перевага першої структури. Якщо оцінка від'ємна – це перевага другої структури. Оцінка R- демонструє співвідношення областей переваги, а саме вказує на перевагу першої структури над другою. Якщо оцінка дорівнює 50% – області переваг рівнозначні.

Теоретичні відомості для поглибленого вивчення

1. *Обґрунтування методу обчислення S-, R- оцінок.*

Розглянемо статистичні показники часової ефективності алгоритмів операцій обробки структур даних. Такі показники враховують усі фактори, що

впливають на час виконання алгоритму. Такими факторами є вхідні дані, програмно-апаратні середовища підготовки й виконання алгоритмів.

Часова ефективність алгоритму визначається функцією

$$E = f(v,t,x,\psi,r), \tag{16}$$

де v – обсяг, t – тип, x – значення вхідних даних, ψ – програмне середовище створення та функціонування exe- файлу, r – архітектура ЕОМ.

За наявності декількох функціонально еквівалентних алгоритмів слід порівняти функції

$$E_1 = f_1(v,t,x,\psi,r), \quad E_2 = f_2(v,t,x,\psi,r), \ldots \quad E_n = f_n(v,t,x,\psi,r). \tag{17}$$

Нехай V^* – множина можливих значень обсягу даних, T^* – множина можливих типів даних, X^* – множина можливих значень даних, Ψ^* – множина можливих програмних середовищ, $\Re^*$ – множина архітектур ЕОМ, на яких можлива реалізація алгоритмів. Тоді для порівняння часової ефективності альтернативних обчислювальних алгоритмів необхідно визначити такі показники:

– ступінь переваги одного (i-го) алгоритму над іншим (j-м) на обмежених множинах, що є підмножинами зазначених вище множин $\bar{V} \subseteq V^*$, $\bar{T} \subseteq T^*$, $\bar{X} \subseteq X^*$, $\bar{\Psi} \subseteq \Psi^*$, $\bar{\Re} \subseteq \Re^*$, як функціонал:

$$SUP_{ij}\left|\bar{V},\bar{T},\bar{X},\bar{\Psi},\bar{\Re} = \frac{1}{N}\sum_{v\in\bar{V}}\sum_{t\in\bar{T}}\sum_{x\in\bar{X}}\sum_{\psi\in\bar{\Psi}}\sum_{r\in\bar{\Re}} g_{ij}(v,t,x,\psi,r),\right. \tag{18}$$

де N – загальна кількість доданків, g_{ij} – ступінь переваги i-го алгоритму над j-м в точці, яка визначається як

$$g_{ij}(v,t,x,\psi,r) = \frac{f_j(v,t,x,\psi,r) - f_i(v,t,x,\psi,r)}{\max(f_i(v,t,x,\psi,r), f_j(v,t,x,\psi,r))}, \tag{19}$$

де f_i – час виконання i-го алгоритму;

– область переваги:

$$REG_{ij}\left|\bar{V},\bar{T},\bar{X},\bar{\Psi},\bar{\Re} = \frac{1}{N}\sum_{v\in\bar{V}}\sum_{t\in\bar{T}}\sum_{x\in\bar{X}}\sum_{\psi\in\bar{\Psi}}\sum_{r\in\bar{\Re}} h_{ij}(v,t,x,\psi,r),\right. \tag{20}$$

де h_{ij} – ознака переваги i-го алгоритму над j-м у точці:

$$h_{ij}(v,t,x,\psi,r) = sign(f_j(v,t,x,\psi,r) - f_i(v,t,x,\psi,r)), \quad sign(a) = \begin{cases} 1, \textit{ при } a > 0 \\ 0, \textit{ при } a \le 0 \end{cases}; \tag{21}$$

Вважаючи, що множини $\bar{V}, \bar{T}, \bar{X}, \bar{\Psi}, \bar{\Re}$ – дискретні й обмежені, їх можна відобразити на множину натуральних чисел $\bar{V} \to \bar{\bar{V}} = \{1,2, \ldots N_1\}$, ... $\bar{\Re} \to \bar{\bar{\Re}} = = \{1,2, \ldots N_5\}$ відповідно, при цьому:

$$\hat{g}_{ij}(\bar{\bar{v}},\bar{\bar{t}},\bar{\bar{x}},\bar{\bar{\psi}},\bar{\bar{r}}) = g_{ij}(v,t,x,\psi,r). \tag{22}$$

Оцінку показників виконуємо за допомогою методу Монте-Карло, для чого перейдемо від суми до інтегрування ступінчастої функції.

В області $\Omega = \tilde{V} \times \tilde{T} \times \tilde{X} \times \tilde{\Psi} \times \tilde{\Re} = [1 \dots N_1 + 1] \times [1 \dots N_2 + 1] \times \dots \times [1 \dots N_5 + 1]$ визначимо:

$$\tilde{g}_{ij}(\tilde{v}, \tilde{t}, \tilde{x}, \tilde{\psi}, \tilde{r}) = \hat{g}_{ij}(\bar{\bar{v}}, \bar{\bar{t}}, \bar{\bar{x}}, \bar{\bar{\psi}}, \bar{\bar{r}}), \tag{23}$$

де $\tilde{v} \in [\bar{\bar{v}}, \bar{\bar{v}} + 1)$, $\tilde{t} \in [\bar{\bar{t}}, \bar{\bar{t}} + 1) \dots \tilde{r} \in [\bar{\bar{r}}, \bar{\bar{r}} + 1)$, і замінимо суму інтегралом:

$$SUP_{ij}\left|\bar{V}, \bar{T}, \bar{X}, \bar{\Psi}, \bar{\Re} = \frac{1}{N} \sum_{v \in \bar{V}} \sum_{t \in \bar{T}} \sum_{x \in \bar{X}} \sum_{\psi \in \bar{\Psi}} \sum_{r \in \bar{\Re}} g_{ij}(v, t, x, \psi, r) = I = \right.$$

$$= \frac{1}{\prod_{m=1}^{5} N_m} \int_1^{N_1+1} \int_1^{N_2+1} \int_1^{N_3+1} \int_1^{N_4+1} \int_1^{N_5+1} \tilde{g}_{ij}(\tilde{v}, \tilde{t}, \tilde{x}, \tilde{\psi}, \tilde{r}) d\tilde{v} d\tilde{t} d\tilde{x} d\tilde{\psi} d\tilde{r}. \tag{24}$$

Якщо $\bar{V}$ або $\bar{X}$ є безперервними, відповідні суми в (18), (19) і (21) спочатку замінюються інтегралами і відпадає необхідність у переході до (24).

В області Ω виберемо випадковим чином $\bar{N}$ точок P=$(\tilde{v}, \tilde{t}, \tilde{x}, \tilde{\psi}, \tilde{r})$ і обчислимо значення випадкової величини $\xi = \tilde{g}_{ij}(\tilde{v}, \tilde{t}, \tilde{x}, \tilde{\psi}, \tilde{r}) = g_{ij}(v, t, x, \psi, r)$. Інтеграл (24) при цьому є математичним сподіванням випадкової величини ξ, тобто $M[\xi] = I$.

Як відомо, оцінкою математичного сподівання є величина

$$\theta_S = \frac{1}{\bar{N}} \sum_{k=1}^{\bar{N}} \xi_k = \frac{1}{\bar{N}} \sum_{k=1}^{\bar{N}} g_{ijk}(v, t, x, \psi, r). \tag{25}$$

S- оцінку ступеня переваги *i*-го алгоритму над *j*-м (18) визначимо як

$$S_{ij}\left|\bar{V}, \bar{T}, \bar{X}, \bar{\Psi}, \bar{\Re} = \theta_S \cdot 100\% = \frac{1}{\bar{N}} \sum_{k=1}^{\bar{N}} g_{ijk}(v, t, x, \psi, r) \cdot 100\%. \right. \tag{26}$$

Оцінка довірчих інтервалів проводиться наступним чином:

$$\theta_S - t_{2,\beta} \sqrt{\frac{1}{6} \sum_{k=1}^{3} (\zeta_k - \theta_S)^2} \cdot 100 < S_{ij}\left|\bar{V}, \bar{T}, \bar{X}, \bar{\Psi}, \bar{\Re}, \beta \right. \text{ та}$$

$$S_{ij}\left|\bar{V}, \bar{T}, \bar{X}, \bar{\Psi}, \bar{\Re}, \beta < \theta_s + t_{2,\beta} \sqrt{\frac{1}{6} \sum_{k=1}^{3} (\zeta_k - \theta_S)^2} \cdot 100. \right. \tag{27}$$

Значення $g_{ijk}(v, t, x, \psi, r)$ визначаються експериментально.

Підставивши (19) у (26) отримаємо S-оцінку

$$S = \frac{1}{N} \sum \frac{f_j(x) - f_i(x)}{\max(f_i(x), f_j(x))} \cdot 100\% \tag{28}$$

Аналогічно:

$$R = \frac{1}{N} \sum sign(f_j(x) - f_i(x)) \cdot 100\%, \;\; sign(a) = \begin{cases} 1, \text{ якщо } a > 0 \\ 0, \text{ якщо } a \le 0 \end{cases}; \tag{29}$$

2. Використання бібліотеки шаблонів STL.

У лабораторних роботах операції обробки структур даних розроблюються самостійно. Але існують стандартні напрацювання, наприклад, контейнери із стандартної бібліотеки шаблонів (STL). Ця бібліотека містить набір узгодже-

них контейнерів та алгоритмів для роботи з ними. Серед контейнерів реалізовані наступні структури даних: вектор (масив), двохзв'язний список, стек, черга і т.п.

Вектор і список з бібліотеки, за замовчуванням, не підтримують сортування доданих значень. Підтримку впорядкованості контейнера можна додати, використовуючи сімейство функцій для бінарного пошуку: upper_bound, lower_bound, binary_search. У нашому випадку може бути використана функція lower_bound, яка призначена для пошуку місця в контейнері, куди можна вставити шукане значення. Потім, саме значення додається до контейнеру за допомогою функції вставки.

```
#include <list>
#include <algorithm>
List list;
std::list<int> stlList;
list.Insert(data_to_add);
// Пошук позиції для вставки. Підтримка відсортованого списку.
std::list<int>::iterator                insertIt                =
std::lower_bound(stlList.begin(), stlList.end(), data_to_add);
stlList.insert(insertIt, data_to_add);
```

```
#include <vector>
Array array;
std::vector<int> stlVector;
array.Insert(data_to_add);
// Пошук позиції для вставки. Підтримка відсортованого массиву.
std::vector<int>::iterator                insertIt                =
std::lower_bound(stlVector.begin(),stlVector.end(),
data_to_add);
stlVector.push_back(data_to_add);
```

Завдання

На основі результатів лабораторної роботи №3, вибрати дві структури даних для зберігання лексем програми.

Для тексту програми на заданій мові програмування (за матеріалами лабораторної роботи № 3) виконати пошук всіх лексем класу, визначеного варіантом. Знайдені лексеми занести в структури даних.

Виконати операції обробки даних з виміром часу. Для цього заповнити структури, розроблені в л.р. 3, лексемами в кількості a шт, де *a* - випадкове число, яке не перевищує загальну кількість знайдених лексем заданого класу в програмі. Провести серії випробувань за такими сценаріями:

1) додати *(a + 1)*-елемент в структури, виконавши замір часу виконання операції;
2) видалити елемент, який знаходиться в структурі, порядковий номер якого – випадкове число в діапазоні від 0 до *a*, виконавши замір часу виконання операції;
3) знайти елемент, який знаходиться в структурі, порядковий номер якого – випадкове число в діапазоні від 0 до *a*, виконавши замір часу виконання операції.

На основі отриманих даних виконати оцінювання часової ефективності алгоритмів.

Контрольні питання

1. Фактори, які впливають на час виконання алгоритмів.
2. Розрахунок S- та R- показників часової ефективності при порівнянні двох структур даних.
3. Вплив значень вхідних даних на часову ефективність структур даних.
4. Показники часової ефективності алгоритмів.
5. Вплив апаратного середовища на часову ефективність структур даних.
6. Вплив програмного середовища на часову ефективність структур даних.

Контрольні питання підвищеної складності

1. Критерії оцінки довірчого інтервалу.
2. Обґрунтування розрахунку S- та R- показників часової ефективності.
3. Причини можливої розбіжності висновків за показниками обчислювальної складності та часової ефективності.
4. Методи визначення показників часової ефективності.
5. Оцінка часової ефективності методів доступу до даних, наданих середовищами програмування.
6. Обмеження застосування методів оцінки обчислювальної складності.
7. Обмеження застосування методів оцінки часової ефективності.
8. Класи задач, для розв’язання яких призначені показники часової ефективності алгоритмів.

ОСНОВНА ЛІТЕРАТУРА

1. Кнут Д. Э. Искусство программирования, том 3. Сортировка и поиск / Д. Кнут. – [3-е изд.]. – М.: Издательский дом "Вильямс", 2000. – 832 с.
2. Кормен Т. Алгоритмы: построение и анализ / Т. Кормен, Ч. Лейзерсон, Р. Ривест. – М.: МЦНМО, 2001. – 960 с.
3. Шинкаренко В. И. Экспериментальные исследования алгоритмов в программно-аппаратных средах / В.И. Шинкаренко. – Д.: Изд-во Днепропетр. нац. ун-та железнодор. трансп. им. акад. В. Лазаряна, 2009. – 279 с.
4. Макконнелл Дж. Анализ алгоритмов. Вводный курс / Дж. Макконнелл. – М.: Техносфера, 2002. – 304 с.
5. Маерс Г. Искусство тестирования программ / Пер. с анг. Гузикевич А. – М.: Вильямс, 2012. – 272 с., ил.

Підписано до друку 04.10.2018. Формат паперу . 60×84 1/16
Ум. друк. арк. 2,14. Обл.-вид. арк. 2,34. Наклад 20 пр. Зам. № 47.

Видавництво ПФ «Стандарт-Сервіс»
Свідоцтво ДК № 3197 від 28.05.2008 р.
52005, Україна, Дніпропетровська обл., смт Слобожанське,
вул. Василя Сухомлинського, 68, кв. 65